

2. TỔNG QUAN VỀ TẬP DỮ LIỆU

2.1. NGUỒN DỮ LIỆU

Tập dữ liệu "house prices" được lấy từ trang web Kaggle, một trang web cung cấp các bộ dữ liệu và bài viết về nhiều chủ đề đa dạng khác nhau. Đường dẫn trực tiếp đến trang tải xuống tập dữ liệu về House Prices: [House Prices - Advanced Regression Techniques | Kaggle](#)

2.2. CHI TIẾT TẬP DỮ LIỆU

Tập dữ liệu House Prices bao gồm thông tin về giá cả và thông tin của các ngôi nhà đã được thực hiện giao dịch. Dữ liệu bao gồm 81 cột và 1460 dòng.

- ***MSSubClass***: Xác định loại nhà ở liên quan đến việc bán

20	1-STORY 1946 & NEWER ALL STYLES
30	1-STORY 1945 & OLDER
40	1-STORY W/FINISHED ATTIC ALL AGES
45	1-1/2 STORY - UNFINISHED ALL AGES
50	1-1/2 STORY FINISHED ALL AGES
60	2-STORY 1946 & NEWER
70	2-STORY 1945 & OLDER
75	2-1/2 STORY ALL AGES
80	SPLIT OR MULTI-LEVEL
85	SPLIT FOYER
90	DUPLEX - ALL STYLES AND AGES
120	1-STORY PUD (Planned Unit Development) - 1946 & NEWER
150	1-1/2 STORY PUD - ALL AGES
160	2-STORY PUD - 1946 & NEWER

- **MSZoning:** Xác định phân loại quy hoạch chung của giao dịch bán

A	Nông nghiệp
C	Thương mại
FV	Khu dân cư làng nổi
I	Công nghiệp
RH	Khu dân cư mật độ cao
RL	Khu dân cư mật độ thấp
RP	Khu dân cư mật độ thấp - Công viên
RM	Khu dân cư mật độ trung bình

- **LotFrontage:** Chiều dài tuyến đường nối với tài sản (feet)

- **LotArea:** Diện tích lô đất (feet vuông)

- **Street:** Loại đường vào khu đất

Grvl	Đường Sỏi
Pave	Đường Đã trải nhựa

- **Alley:** Loại ngõ/hẻm vào khu đất

Grvl	Sỏi
Pave	Đã trải nhựa
NA	Không có đường vào ngõ

- **LotShape:** Hình dạng chung của khu đất

Reg	Hình dạng chuẩn (Regular)
IR1	Hình dạng hơi bất thường (Slightly irregular)
IR2	Hình dạng khá bất thường (Moderately Irregular)
IR3	Hình dạng bất thường (Irregular)

- **LandContour:** Độ bằng phẳng của khu đất

Lvl	Bằng phẳng
Bnk	Dốc - Nhanh chóng và dốc đáng kể từ mặt đường lên tòa nhà
HLS	Sườn đồi - Độ dốc đáng kể từ bên này sang bên kia
Low	Trũng

- **Utilities:** Loại tiện ích sẵn có

AllPub	Tất cả các tiện ích công cộng (Điện, Gas, Nước & Thoát nước)
NoSewr	Điện, Gas và Nước
NoSeWa	Chỉ có Điện và Gas
ELO	Chỉ có Điện

- **LotConfig:** Cấu hình lô đất

Inside	Lô đất bên trong
Corner	Lô đất góc
CulDSac	Cul-de-sac
FR2	Mặt tiền 2 phía
FR3	Mặt tiền 3 phía

- ***LandSlope***: Độ dốc của khu đất

Gtl Dốc nhẹ

Mod Dốc vừa

Sev Dốc cao

- ***Neighborhood***: Các vị trí thực tế trong giới hạn thành phố Ames

Blmngtn	Bloomington Heights
Blueste	Bluestem
BrDale	Briardale
BrkSide	Brookside
ClearCr	Clear Creek
CollgCr	College Creek
Crawfor	Crawford
Edwards	Edwards
Gilbert	Gilbert
IDOTRR	Iowa DOT and Rail Road
MeadowV	Meadow Village
Mitchel	Mitchell
Names	North Ames
NoRidge	Northridge
NPkVill	Northpark Villa
NridgHt	Northridge Heights
NWAmes	Northwest Ames

OldTown	Old Town
SWISU	South & West of Iowa State University
Sawyer	Sawyer
SawyerW	Sawyer West
Somerst	Somerset
StoneBr	Stone Brook
Timber	Timberland
Veenker	Veenker

- **Condition1:** Vị trí gần các yếu tố khác nhau

Artery	Gần đường giao thông chính/ đường lớn
Feedr	Gần đường nhánh/ đường phụ
Norm	Bình thường
RRNn	Cách đường sắt Bắc-Nam trong phạm vi 200'
RRAn	Gần đường sắt Bắc-Nam
PosN	Gần tiện ích tích cực bên ngoài - công viên, vành đai xanh, ...
PosA	Gần tiện ích tích cực bên ngoài
RRNe	Cách đường sắt Đông-Tây trong phạm vi 200'
RR Ae	Gần đường sắt Đông-Tây

- **Condition2:** Vị trí gần các yếu tố khác nhau (Nếu nhiều hơn 1)

Artery	Gần đường giao thông chính
Feedr	Gần đường nhánh
Norm	Bình thường

RRNn Cách đường sắt Bắc-Nam trong phạm vi 200'

RRAn Gần đường sắt Bắc-Nam

PosN Gần tiện ích tích cực bên ngoài - công viên, vành đai xanh, ...

PosA Gần tiện ích tích cực bên ngoài

RRNe Cách đường sắt Đông-Tây trong phạm vi 200'

RRAe Gần đường sắt Đông-Tây

- **BldgType:** Loại nhà ở

1Fam	Nhà đơn lập
2FmCon	Nhà chuyển đổi hai gia đình; ban đầu xây dựng như nhà một gia đình
Duplx	Duplex
TwnhsE	Nhà liền kề, căn góc
TwnhsI	Nhà liền kề, căn giữa

- **HouseStyle:** Phong cách nhà ở

1Story Một tầng

1.5Fin Một tầng rưỡi: Tầng 2 hoàn thiện

1.5Unf Một tầng rưỡi: Tầng 2 chưa hoàn thiện

2Story Hai tầng

2.5Fin Hai tầng rưỡi: Tầng 2 hoàn thiện

2.5Unf Hai tầng rưỡi: Tầng 2 chưa hoàn thiện

SFoyer Nhà phân tầng

SLvl Nhà phân tầng

- **OverallQual:** Đánh giá tổng thể vật liệu và độ hoàn thiện của căn nhà

10	Rất tuyệt vời
9	Tuyệt vời
8	Rất tốt
7	Tốt
6	Trên mức trung bình
5	Trung bình
4	Dưới trung bình
3	Tạm ổn
2	Tệ
1	Rất tệ

- **OverallCond:** Đánh giá tình trạng chung của căn nhà

10	Rất tuyệt vời
9	Tuyệt vời
8	Rất tốt
7	Tốt
6	Trên trung bình
5	Trung bình
4	Dưới trung bình
3	Tạm ổn
2	Tệ
1	Rất tệ

- ***YearBuilt***: Năm khởi công xây dựng
- ***YearRemodAdd***: Năm sửa chữa/cải tạo (going năm xây dựng nếu không sửa chữa, bổ sung)

- ***RoofStyle***: Loại mái nhà

Flat	Mái phẳng
Gable	Mái dốc
Gambrel	Mái chuồng (Barn)
Hip	Mái chóp
Mansard	Mansard
Shed	Mái che

- ***RoofMatl***: Vật liệu mái nhà

ClyTile	Ngói hoặc gạch
CompShg	Ngói tổ hợp tiêu chuẩn
Membran	Màng
Metal	Kim loại
Roll	Cuộn
Tar&Grv	Lớp nhựa & sỏi
WdShake	Tấm lợp gỗ
WdShngl	Ngói gỗ

- ***Exterior1st***: Lớp phủ bên ngoài ngôi nhà

AsbShng	Ngói amiăng
---------	-------------

AsphShn	Ngói nhựa đường
BrkComm	Gạch thường
BrkFace	Mặt gạch
CBlock	Khối xi măng
CemntBd	Ván xi măng
HdBoard	Ván cứng
ImStucc	Vữa giả
MetalSd	Tấm kim loại
Other	Khác
Plywood	Ván ép
PreCast	Đúc sẵn
Stone	Đá
Stucco	Vữa
VinylSd	Tấm vinyl
Wd Sdng	Tấm gỗ
WdShing	Ngói gỗ

- ***Exterior2nd:*** Lớp phủ bên ngoài ngôi nhà (Nếu có nhiều hơn 1 nguyên liệu)

AsbShng	Ngói amiăng
AsphShn	Ngói nhựa đường
BrkComm	Gạch thường
BrkFace	Mặt gạch
CBlock	Khối xi măng
CemntBd	Ván xi măng

HdBoard	Ván cứng
ImStucc	Vữa giả
MetalSd	Tấm kim loại
Other	Khác
Plywood	Ván ép
PreCast	Đúc sẵn
Stone	Đá
Stucco	Vữa
VinylSd	Tấm vinyl
Wd Sdng	Tấm gỗ
WdShing	Ngói gỗ

- ***MasVnrType:*** Loại tấm ốp gạch

BrkCmn	Gạch thường
BrkFace	Mặt gạch
CBlock	Khối xi măng
None	Không có
Stone	Đá

- ***MasVnrArea:*** Diện tích tấm ốp gạch (feet vuông)

- ***ExterQual:*** Đánh giá chất lượng vật liệu bên ngoài

Ex	Tuyệt vời
Gd	Tốt

TA Trung bình

Fa Tạm ổn

Po Tệ

- **ExterCond:** Đánh giá hiện trạng bên ngoài của vật liệu

Ex Tuyệt vời

Gd Tốt

TA Trung bình

Fa Tạm ổn

Po Tệ

- **Foundation:** Loại móng nhà

BrkTil Gạch & Ngói

CBlock Khối xi măng

PConc Bê tông đổ

Slab Bản mỏng

Stone Đá

Wood Gỗ

- **BsmtQual:** Đánh giá chiều cao tầng hầm

Ex Tuyệt vời (100+ inches)

Gd Tốt (90-99 inches)

TA Trung bình (80-89 inches)

Fa Tạm ổn (70-79 inches)

Po Tệ (<70 inches)
NA Không có tầng hầm

• **BsmtCond:** Đánh giá tình trạng chung của tầng hầm

Ex Tuyệt vời
Gd Tốt
TA Trung bình - cho phép độ ẩm nhẹ
Fa Độ ẩm hoặc một số vết nứt hoặc lắng đọng - Tạm ổn
Po Vết nứt, lắng đọng hoặc ẩm ướt nghiêm trọng - Tệ
NA Không có tầng hầm

• **BsmtExposure:** Đề cập đến các bức tường lối đi bộ hoặc lối đi trong vườn

Gd Độ phơi sáng tốt
Av Độ phơi sáng trung bình - (các tầng chia tách hoặc tiền sảnh thường đạt điểm trung bình trở lên)
Mn Độ phơi sáng tối thiểu
No Không có ánh sáng
NA Không có tầng hầm

• **BsmtFinType1:** Đánh giá diện tích hoàn thiện tầng hầm của loại 1

GLQ Khu nhà ở tốt
ALQ Khu nhà ở trung bình
BLQ Khu nhà ở dưới mức trung bình
Rec Phòng giải trí trung bình
LwQ Chất lượng thấp

Unf Chưa hoàn thiện
NA Không có tầng hầm

- **BsmtFinSF1:** Diện tích hoàn thiện tầng hầm của loại 1 (feet vuông)
- **BsmtFinType2:** Đánh giá diện tích hoàn thiện tầng hầm của loại 2 (nếu nhiều loại)

GLQ Khu nhà ở tốt
ALQ Khu nhà ở trung bình
BLQ Khu nhà ở dưới mức trung bình
Rec Phòng giải trí trung bình
LwQ Chất lượng thấp
Unf Chưa hoàn thiện
NA Không có tầng hầm

- **BsmtFinSF2:** Diện tích hoàn thiện tầng hầm của loại 2 (tính theo feet vuông)
- **BsmtUnfSF:** Diện tích tầng hầm chưa hoàn thiện
- **TotalBsmtSF:** Tổng feet vuông diện tích tầng hầm
- **Heating:** Loại sưởi ấm

Floor Lò sán

GasA	Lò sưởi không khí bằng áp suất khí
GasW	Khí nóng hoặc hơi nước nóng
Grav	Lò trọng lực
OthW	Nước nóng hoặc nhiệt hơi nước không phải là gas
Wall	Lò sưởi tường

- ***HeatingQC:*** Chất lượng và tình trạng sưởi ấm

Ex	Tuyệt vời
Gd	Tốt
TA	Trung bình
Fa	Tạm ổn
Po	Tệ

- ***CentralAir:*** Điều hòa không khí

NKhông
Y Có

- ***Electrical:*** Hệ thống điện

SBrkr	Cầu dao tiêu chuẩn & Romex
FuseA	Hộp cầu chì trên 60 AMP và tất cả dây Romex (Trung bình)
FuseF	Hộp cầu chì 60 AMP và chủ yếu dây Romex (Tạm ổn)
FuseP	Hộp cầu chì 60 AMP và chủ yếu dây cũ & ống (Tệ)
Mix	Hỗn hợp

- ***1stFlrSF***: Diện tích Tầng 1 (feet vuông)
- ***2ndFlrSF***: Diện tích tầng 2 (feet vuông)
- ***LowQualFinSF***: Chất lượng hoàn thiện thấp (tất cả các tầng) - (feet vuông)
- ***GrLivArea***: Diện tích sinh hoạt trên mặt đất (feet vuông)
- ***BsmtFullBath***: Phòng tắm đầy đủ nội thất trong tầng hầm
- ***BsmtHalfBath***: Phòng tắm một nửa nội thất trong tầng hầm
- ***FullBath***: Phòng tắm đầy đủ nội thất trên mặt đất
- ***HalfBath***: Phòng tắm một nửa nội thất trên mặt đất
- ***Bedroom***: Số lượng phòng ngủ trên mặt đất (KHÔNG bao gồm phòng ngủ ở tầng hầm)
- ***Kitchen***: Nhà bếp trên tầng
- ***KitchenQual***: Chất lượng nhà bếp
Ex Tuyệt vời
Gd Tốt

TA Trung bình

Fa Tạm ổn

Po Tệ

- **TotRmsAbvGrd:** Tổng số phòng trên mặt đất (không bao gồm phòng tắm).

- **Functional:** Chức năng nhà (giả định chức năng điển hình trừ khi có sự giảm trừ).

Typ Chức năng điển hình

Min1 Giảm nhẹ 1

Min2 Giảm nhẹ 2

Mod Giảm vừa

Maj1 Giảm chính 1

Maj2 Giảm chính 2

Sev Hư hỏng nghiêm trọng

Sal Chỉ dùng để cứu hộ

- **Fireplaces:** Số lượng lò sưởi

- **FireplaceQu:** Chất lượng lò sưởi

Ex Tuyệt vời - Lò sưởi xây bằng đá chất lượng ngoại hạng

Gd Tốt - Lò sưởi xây bằng đá ở tầng chính

TA Trung bình - Lò sưởi đúc sẵn ở phòng sinh hoạt chính hoặc Lò sưởi xây bằng đá ở tầng hầm

Fa Tạm ổn - Lò sưởi đúc sẵn ở tầng hầm

Po Tê - Lò sưởi Ben Franklin

NA Không lò sưởi

- **GarageType:** Vị trí gara

2Types Hơn một loại ga ra

Attchd Nhà để xe gắn liền

Basment Ga ra tầng hầm

BuiltIn Ga ra tích hợp - Built-In (Ga ra là một phần của ngôi nhà - thường có phòng trên ga ra)

CarPort Chỗ đỗ xe có mái che

Detchd Nhà để xe riêng biệt

NA Không garage

- **GarageYrBltd:** Năm xây dựng gara

- **GarageFinish:** Độ Hoàn thiện nội thất gara

Fin Hoàn thiện

RFn Hoàn thiện tương đối

Unf Chưa hoàn thiện

NA Không garage

- **GarageCars:** Sức chứa gara (số lượng xe).

- **GarageArea:** Diện tích gara (feet vuông).

- ***GarageQual:*** Chất lượng Garage

Ex	Tuyệt vời
Gd	Tốt
TA	Trung bình
Fa	Tạm ổn
Po	Tệ
NA	Không garage

- ***GarageCond:*** Tình trạng Garage

Ex	Tuyệt vời
Gd	Tốt
TA	Trung bình
Fa	Tạm ổn
Po	Tệ
NA	Không garage

- ***PavedDrive:*** Đường lái xe trải nhựa

Y	Đã trải nhựa
P	Trải nhựa một phần
N	Bụi bẩn/sỏi

- ***WoodDeckSF:*** Diện tích sàn gỗ tính bằng feet vuông

- ***OpenPorchSF***: Diện tích hiên mở tính bằng feet vuông
- ***EnclosedPorch***: Diện tích hiên nhà kèm theo mét vuông
- ***3SsnPorch***: Diện tích hiên nhà ba mùa tính bằng mét vuông
- ***ScreenPorch***: Màn hình diện tích hiên nhà tính bằng feet vuông
- ***PoolArea***: Diện tích hồ bơi tính bằng feet vuông

- ***PoolQC***: Chất lượng hồ bơi

Ex Tuyệt vời

Gd Tốt

TA Trung bình

Fa Tạm ổn

NA Không có hồ bơi

- ***Fence***: Chất lượng hàng rào

GdPrv Riêng tư tốt

MnPrv Riêng tư hạn chế

GdWo Chất liệu gỗ tốt

MnWw Gỗ/Dây tối thiểu

NA Không có hàng rào

- **MiscFeature:** Tính năng khác không có trong các danh mục khác

Elev Thang máy

Gar2 Gara thứ 2 (nếu không được mô tả trong phần gara)

Othr Khác

Shed Nhà kho (trên 100 SF)

TenC Sân tennis

NA Không có

- **MiscVal:** Giá trị của tính năng khác (\$)

- **MoSold:** Tháng bán (MM)

- **YrSold:** Năm bán (YYYY)

- **SaleType:** Loại hình giao dịch

WD Giấy chứng nhận quyền sở hữu theo quy ước

CWD Giấy chứng nhận quyền sở hữu - Tiền mặt

VWD Giấy chứng nhận quyền sở hữu - VA Loan

New Nhà mới xây dựng

COD Giấy chuyển nhượng của Tòa án / Di sản

Con Hợp đồng đặt cọc 15% với các điều khoản trả góp thông thường

ConLw Hợp đồng đặt cọc thấp và lãi suất thấp

ConLI Hợp đồng lãi suất thấp

ConLD Hợp đồng đặt cọc thấp

Oth Khác

- ***SaleCondition:*** Điều kiện giao dịch

Normal Giao dịch bình thường

Abnorml Giao dịch bất thường, trao đổi, tịch thu, bán không

AdjLand Mua đất liền kề

Alloca Phân bổ - Hai tài sản được liên kết với các văn bản riêng biệt, thường là căn hộ chung cư với một gara

Family Giao dịch giữa các thành viên gia đình

Partial Ngôi nhà chưa hoàn thành khi được thẩm định giá lần cuối (áp dụng cho Nhà mới)

B. NỘI DUNG

1. Tải và khám phá sơ bộ tập dữ liệu

1.1. Các thư viện cần thiết

```
import numpy as np
import pandas as pd
import datetime
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
import scipy
from scipy.stats import norm
from scipy import stats
from scipy.stats import skew
import sklearn
import warnings
warnings.filterwarnings('ignore')
```

1.2. Tải dữ liệu từ file csv

```
path = './house_input'
df_train = pd.read_csv(path+'/train.csv')
df_test = pd.read_csv(path+'/test.csv')
print ("Data is loaded!")
```

1.3. Xem cấu trúc của tập dữ liệu

```
print ("Train: ", df_train.shape[0], "hàng, và ", df_train.shape[1], "cột")
print ("Test: ", df_test.shape[0], "hàng, và ", df_test.shape[1], "cột")
```

```
Train: 1460 hàng, và 81 cột
Test: 1459 hàng, và 80 cột
```

Tập dữ liệu bao gồm 81 cột, 1460 dòng

Trong đó có 36 thuộc tính là định lượng (quantitative), 43 thuộc tính là phân loại (categorical) + *Id* (đã được loại bỏ) và *SalePrice* mô tả (gần như) mọi khía cạnh của nhà ở ở Ames, Iowa.

- **Định lượng (Quantitative):** *1stFlrSF*, *2ndFlrSF*, *3SsnPorch*, *BedroomAbvGr*, *BsmtFinSF1*, *BsmtFinSF2*, *BsmtFullBath*, *BsmtHalfBath*, *BsmtUnfSF*, *EnclosedPorch*, *Fireplaces*, *FullBath*, *GarageArea*, *GarageCars*, *GarageYrBlt*, *GrLivArea*, *HalfBath*, *KitchenAbvGr*, *LotArea*, *LotFrontage*, *LowQualFinSF*, *MSSubClass*, *MasVnrArea*, *MiscVal*, *MoSold*, *OpenPorchSF*, *OverallCond*, *OverallQual*, *PoolArea*, *ScreenPorch*, *TotRmsAbvGrd*, *TotalBsmtSF*, *WoodDeckSF*, *YearBuilt*, *YearRemodAdd*, *YrSold*
- **Định tính (Qualitative):** *Alley*, *BldgType*, *BsmtCond*, *BsmtExposure*, *BsmtFinType1*, *BsmtFinType2*, *BsmtQual*, *CentralAir*, *Condition1*, *Condition2*, *Electrical*, *ExterCond*, *ExterQual*, *Exterior1st*, *Exterior2nd*, *Fence*, *FireplaceQu*, *Foundation*, *Functional*, *GarageCond*, *GarageFinish*, *GarageQual*, *GarageType*, *Heating*, *HeatingQC*, *HouseStyle*, *KitchenQual*, *LandContour*, *LandSlope*, *LotConfig*, *LotShape*, *MSZoning*, *MasVnrType*, *MiscFeature*, *Neighborhood*, *PavedDrive*, *PoolQC*, *RoofMatl*, *RoofStyle*, *SaleCondition*, *SaleType*, *Street*, *Utilities*.

2. EDA – Phân tích Khám phá Dữ liệu

2.1 Biến mục tiêu – SalePrice

- Thống kê cơ bản

```
print(df_train['SalePrice'].describe())
```

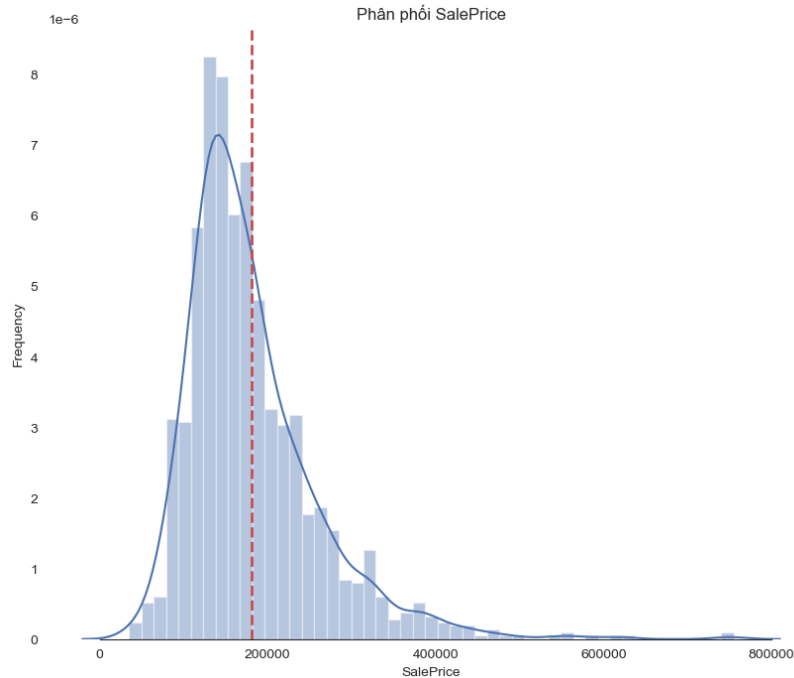
```
count      1460.000000
mean      180921.195890
std       79442.502883
min       34900.000000
25%      129975.000000
50%      163000.000000
75%      214000.000000
max       755000.000000
Name: SalePrice, dtype: float64
```

- Giá nhà trung bình là 180921.195890
- Độ lệch chuẩn (căn bậc 2 phương sai) 79442.502883 cho thấy xung quanh giá trị trung bình có sự dao động khá lớn

- Phân phối giá trị

```
sns.set_style("white")
sns.set_color_codes(palette='deep')
f, ax = plt.subplots(figsize=(10, 8))

# Kiểm tra phân phối
sns.distplot(df_train['SalePrice'], color="b");
ax.axvline(x=df_train['SalePrice'].mean(), color="r", linestyle='--', linewidth=2)
ax.xaxis.grid(False)
ax.set(ylabel="Frequency")
ax.set(xlabel="SalePrice")
ax.set(title="Phân phối SalePrice")
sns.despine(trim=True, left=True)
plt.show()
```

- **Độ lệch**

```
# Skew và kurt  
print("Skewness: %f" % df_train['SalePrice'].skew())  
print("Kurtosis: %f" % df_train['SalePrice'].kurt())
```

Skewness: 1.882876

Kurtosis: 6.536282

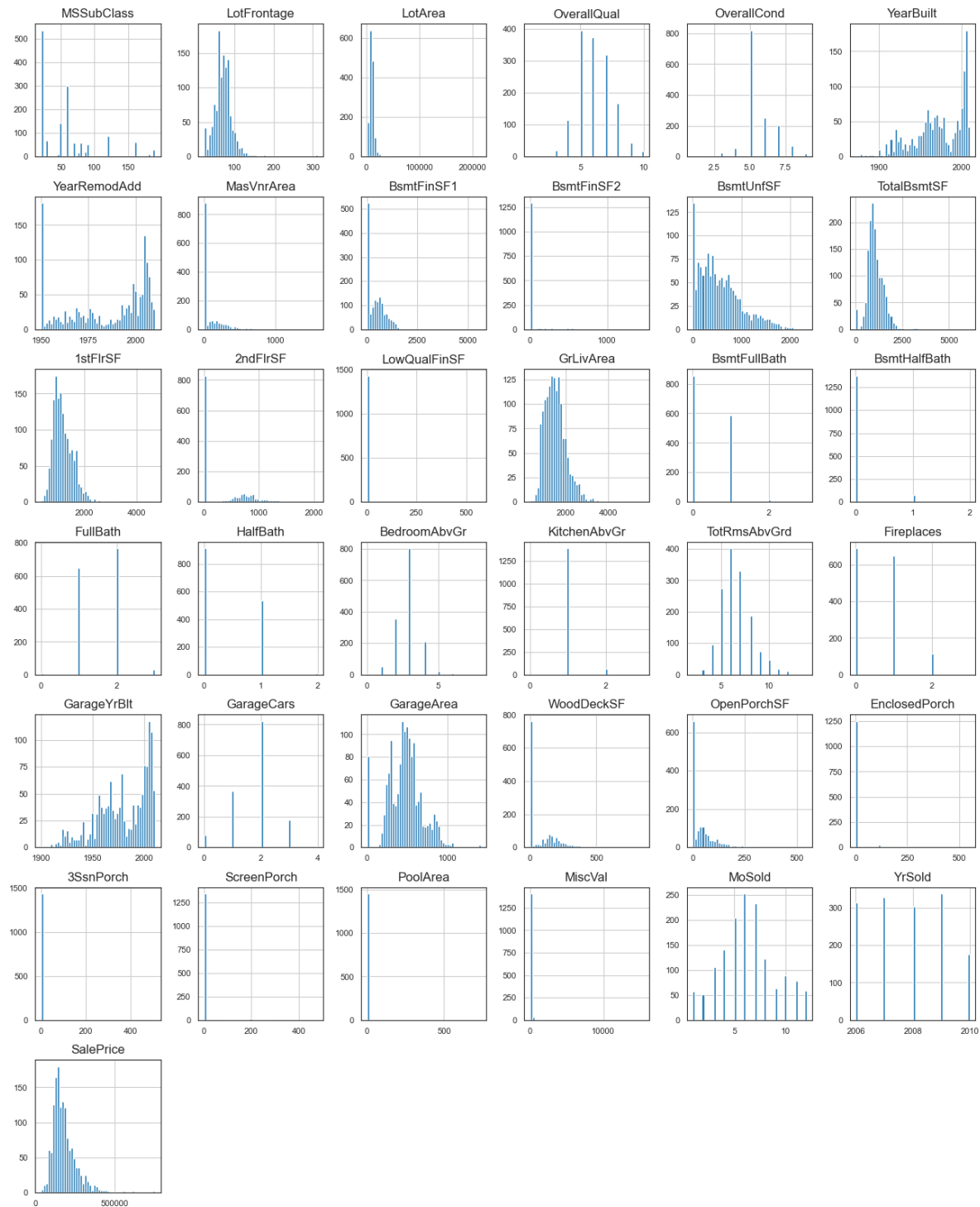
Một số ghi chú từ quan sát phân phối:

- Độ lệch khỏi phân phối chuẩn (normal distribution).
- Có độ lệch chéo dương đáng kể (positive skewness).
- Thể hiện đỉnh cao (show peakedness).

Ta có thể thấy rằng giá bị lệch phải và một số giá trị ngoại lệ nằm trên ~500.000. Vì các mô hình (tuyến tính) ưu tiên dữ liệu được phân phối chuẩn, nên chúng ta cần chuyển đổi biến ***SalePrice*** này và làm cho nó được phân phối chuẩn hơn.

2.2 Các thuộc tính đầu vào (Features)

Có 37 biến số (numeric variable) với biểu đồ phân phối giá trị cho các biến số này như sau:



Các thuộc tính như *1stFlrSF*, *TotalBsmtSF*, *LotFrontage*, *GrLivArea* có phân phối giá trị giống với nhau và giống với mục tiêu *SalePrice*. Ta sẽ thử tìm hiểu sự tương quan giữa các biến này với biến mục tiêu



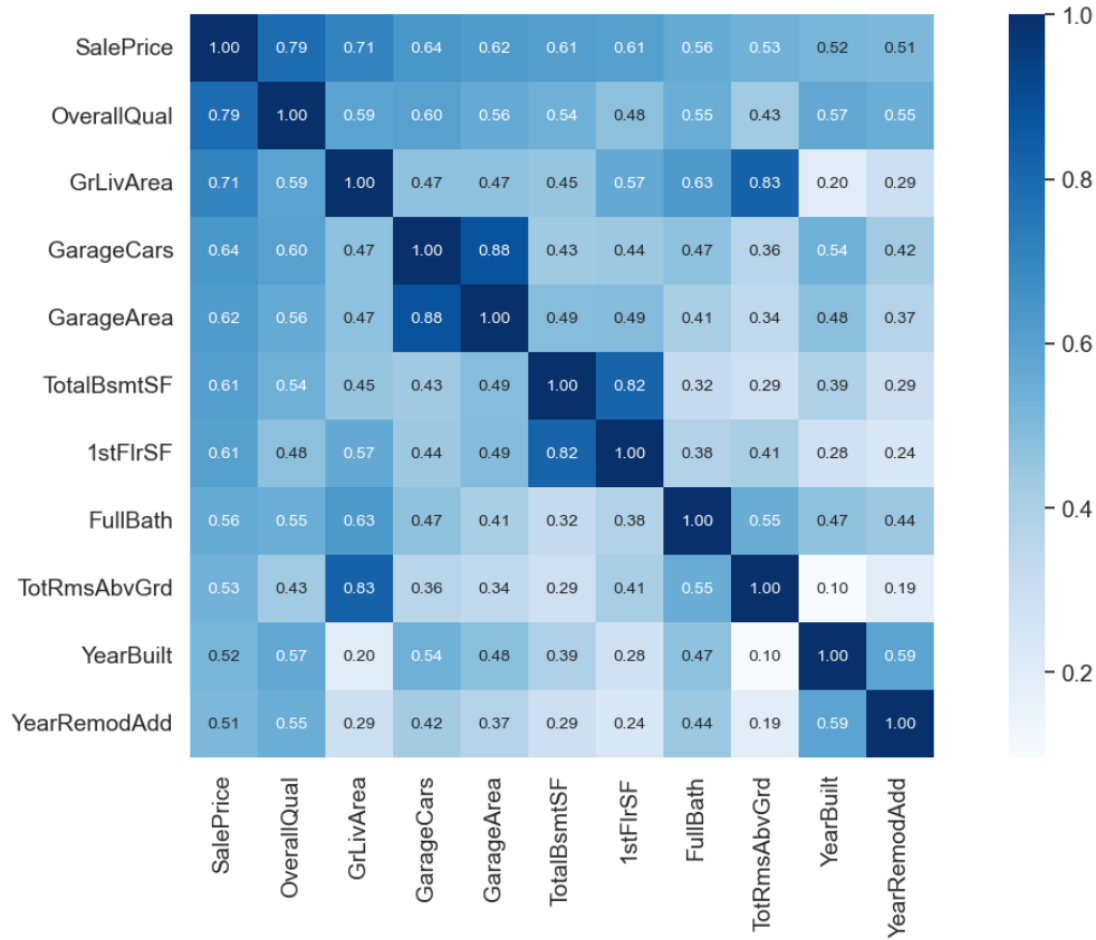
2.3 Sự tương quan (Correlation)

- Ta thấy 2 biến '**TotalBsmtSF**' và '**1stFlrSF**' có độ tương quan cao với nhau. Có thể hiểu bởi việc cả hai đại diện cho các không gian sống chính của ngôi nhà là tầng hầm và tầng trệt, và thường có kích thước tương tự nhau.
- Tuy Các biến **GarageX**(liên quan Garage - GarageArea, GarageCars) cũng có độ tương quan cao với nhau nhưng đều cung cấp thông tin tương tự nhau dẫn đến multicollinearity (đa cộng tuyến). Multicollinearity có thể gây khó khăn trong việc xây dựng mô hình dự đoán chính xác vì nó làm phát sinh vấn đề trong việc ước lượng các hệ số của mô hình.
- Mỗi tương quan giữa **GarageCars** và **GarageArea** rất cao (0.88), và cả hai đều có mối tương quan tương tự (cao) với SalePrice. Tuy nhiên, số xe có thể vừa với diện tích garage là hệ quả của diện tích garage. 'GarageCars' và 'GarageArea' giống như hai anh em sinh đôi. Ta sẽ không thể phân biệt được chúng. Ta sẽ giữ 'GarageCars' vì mối tương quan của nó với 'Saleprice' cao hơn GarageArea, bỏ cột GarageArea. Các biến số khác có mối tương quan cao hơn 0.5 với SalePrice là:
 - + TotalBsmtSF: Tổng diện tích tầng hầm
 - + 1stFlrSF: Diện tích tầng một
 - + FullBath: Phòng tắm đầy đủ phía trên mặt đất
 - + TotRmsAbvGrd: Tổng số phòng phía trên mặt đất (không bao gồm phòng tắm)
 - + YearBuilt: Ngày xây dựng ban đầu
 - + YearRemodAdd: Ngày cải tạo (cùng với ngày xây dựng nếu không có cải tạo hoặc thêm vào)

```

: k = 11 # number of variables/features + SalePrice
cols = corr.nlargest(k, 'SalePrice')['SalePrice'].index
plt.rcParams["figure.figsize"] = (16,8)
cm = np.corrcoef(df_train[cols].values.T)
sns.set(font_scale=1.25)
hm = sns.heatmap(cm, cbar=True, annot=True, square=True, fmt='.2f', annot_kws={'size': 10}, yticklabels=cols.values, xticklabels=cols.values)
plt.show()

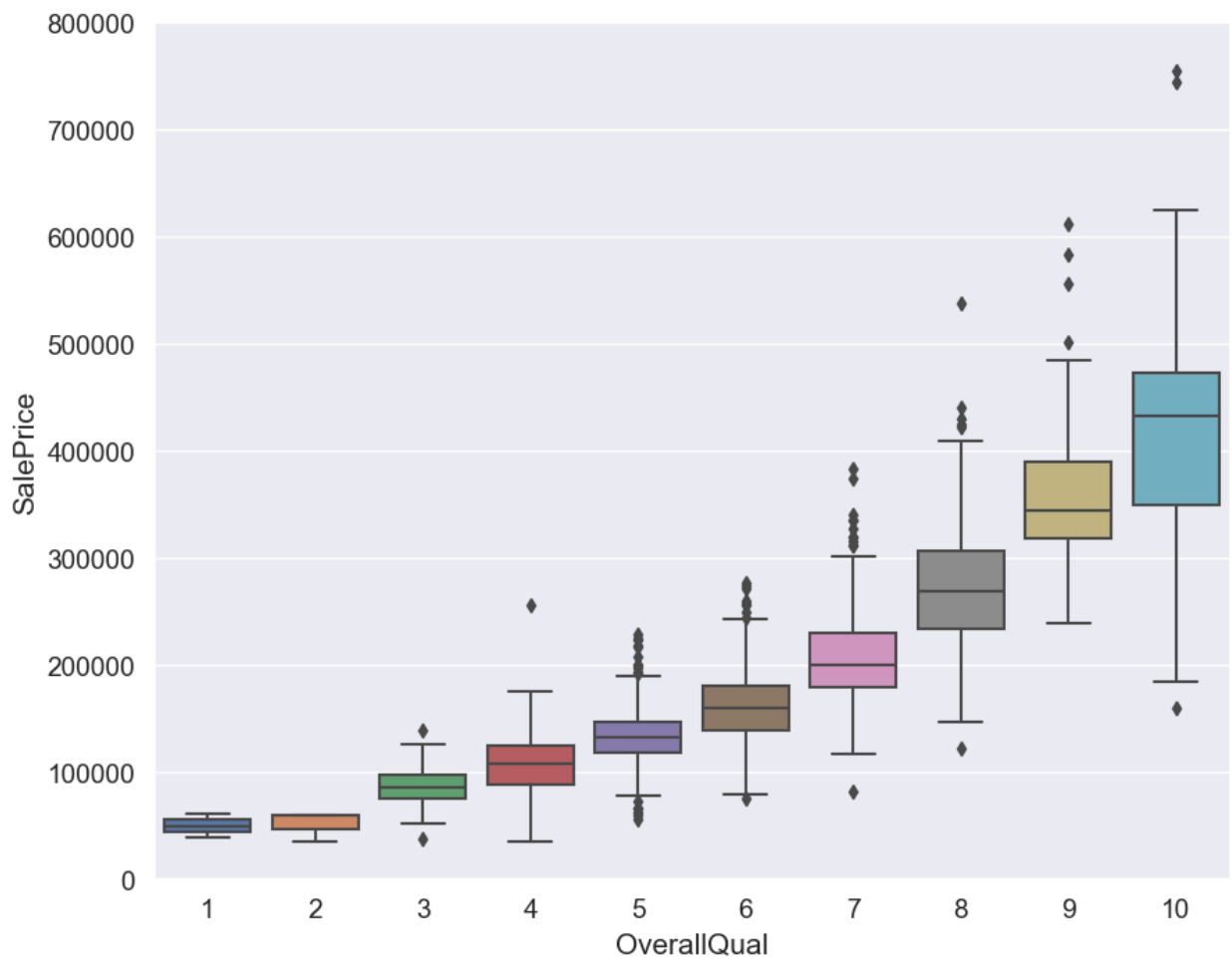
```



2.3.1 Chất lượng tổng thể (OverallQual)

Chất lượng tổng thể có mối tương quan với SalePrice cao nhất trong số các biến số (0,79). Nó đánh giá vật liệu tổng thể và độ hoàn thiện của ngôi nhà theo thang điểm từ 1 (rất kém) đến 10 (rất xuất sắc).

```
#box plot overallqual/saleprice
var = 'OverallQual'
data = pd.concat([df_train['SalePrice'], df_train[var]], axis=1)
f, ax = plt.subplots(figsize=(10, 8))
fig = sns.boxplot(x=var, y="SalePrice", data=data)
fig.axis(ymin=0, ymax=800000);
```



- Mối tương quan tích cực (positive) chắc chắn là có và dường như là một đường cong hơi hướng lên trên. Về các ngoại lệ, chúng tôi không thấy bất kỳ

giá trị cực đoan nào. Nếu sau này có ứng cử viên nào được đưa ra làm ngoại lệ (outlier) thì có vẻ đó là nhà với số điểm 4.

- Với sự tăng của 'OverallQual', giá trung vị và phạm vi giá bán cũng có xu hướng tăng theo. Điều này cho thấy một mối quan hệ tương đối tích cực giữa chất lượng nhà và giá bán.
- Sự phân tán của dữ liệu: IQR mở rộng và sự xuất hiện của ngoại lệ tăng lên với các nhóm chất lượng cao, cho thấy có sự phân tán lớn hơn trong giá bán của nhà có chất lượng cao.
- Ta thấy nếu OverallQual hay chất lượng tổng thể càng cao hay nhà càng được đánh giá cao thì giá nhà SalePrice càng cao

2.3.2 Diện tích sinh hoạt trên mặt đất (GrLivArea)

Biến số có mối tương quan cao thứ hai với SalePrice là Khu vực sinh hoạt trên mặt đất. Rõ ràng, những ngôi nhà lớn thường đắt hơn.

```
# Tạo DataFrame
data = pd.concat([df_train['SalePrice'], df_train['GrLivArea']], axis=1)

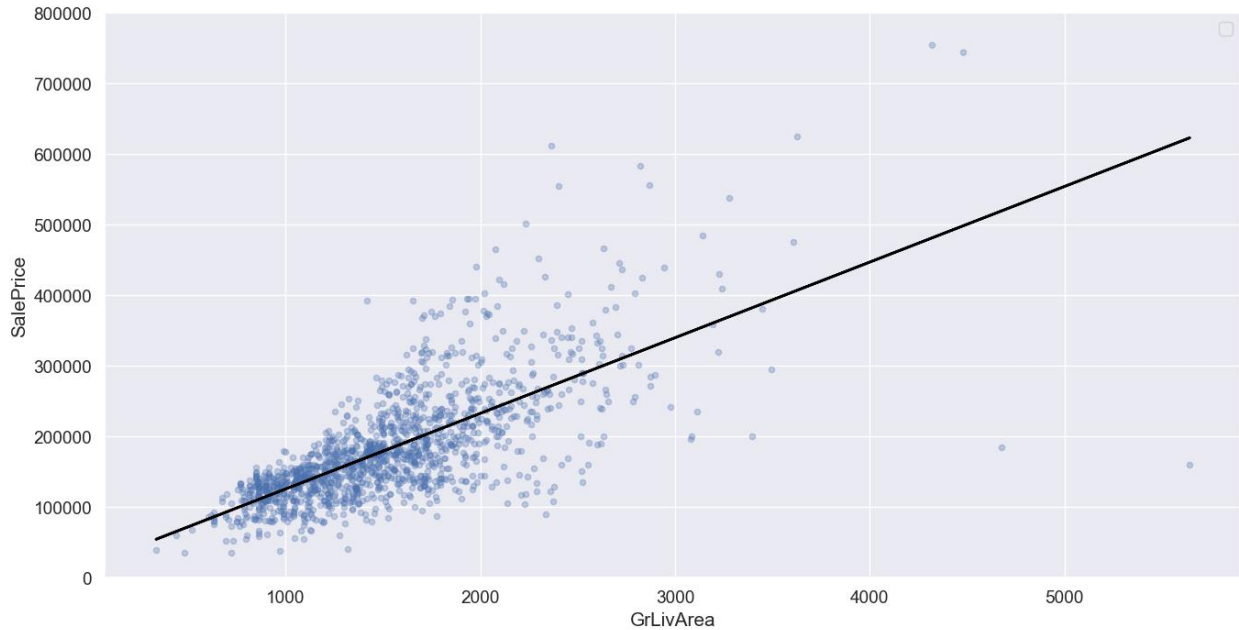
# Vẽ biểu đồ phân tán
plt.figure(figsize=(10, 6)) # Kích thước tùy chọn
ax = data.plot.scatter(x='GrLivArea', y='SalePrice', alpha=0.3, ylim=(0,800000))

# Tính toán hồi quy tuyến tính
x = data['GrLivArea']
y = data['SalePrice']
A = np.vstack([x, np.ones(len(x))]).T
m, c = np.linalg.lstsq(A, y, rcond=None)[0]

# Vẽ đường tuyến tính
plt.plot(x, m*x + c, 'black', linewidth=2)

# Thêm chú thích và hiển thị biểu đồ
plt.legend()
plt.show()
```

Python



Đặc biệt ở đây có hai căn nhà với khu vực sinh hoạt rất rộng và giá bán thấp, có vẻ như là những điểm ngoại lệ. Chúng tôi sẽ chưa loại bỏ chúng ngay bây giờ, vì loại bỏ các điểm ngoại lệ có thể rất nguy hiểm. Chẳng hạn, điểm số thấp về Chất lượng Tổng thể có thể giải thích giá thấp. Tuy nhiên, điểm số Chất lượng Tổng thể cũng có thể cao. Do đó, chúng tôi sẽ tiếp tục để ý đến 2 căn nhà và xem đây là những ứng cử viên hàng đầu để loại bỏ như là các điểm ngoại lệ.

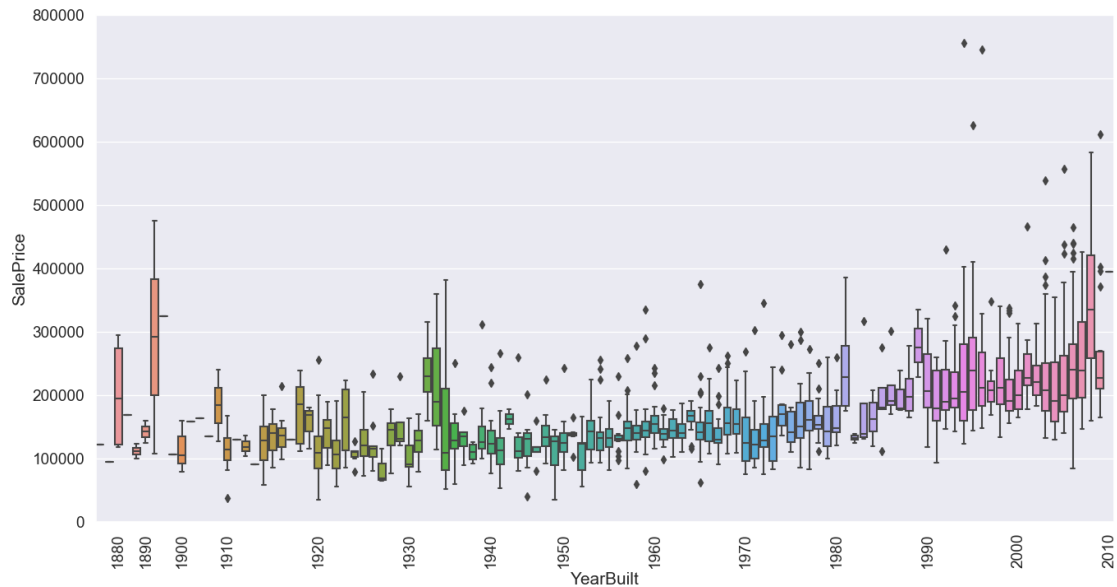
2.3.3 Một vài thuộc tính khác

YearBuilt : Năm xây dựng của ngôi nhà

```
: # Tạo DataFrame
data = pd.concat([df_train['SalePrice'], df_train['YearBuilt']], axis=1)
# Vẽ biểu đồ
f, ax = plt.subplots(figsize=(16, 8))
fig = sns.boxplot(x='YearBuilt', y="SalePrice", data=data)
fig.axis(ymin=0, ymax=800000);

# Điều chỉnh các nhãn trên trục x
# Chỉ hiển thị mỗi năm đầu tiên của mỗi thập kỷ
start_year = 1870
end_year = 2010
step = 10
ticks = [year for year in range(start_year, end_year + 1, step)]
labels = [str(year) if year in ticks else '' for year in sorted(data['YearBuilt'].unique())]

plt.xticks(ticks=range(len(labels)), labels=labels, rotation=90) # Xoay nhãn để dễ đọc hơn
plt.show()
```

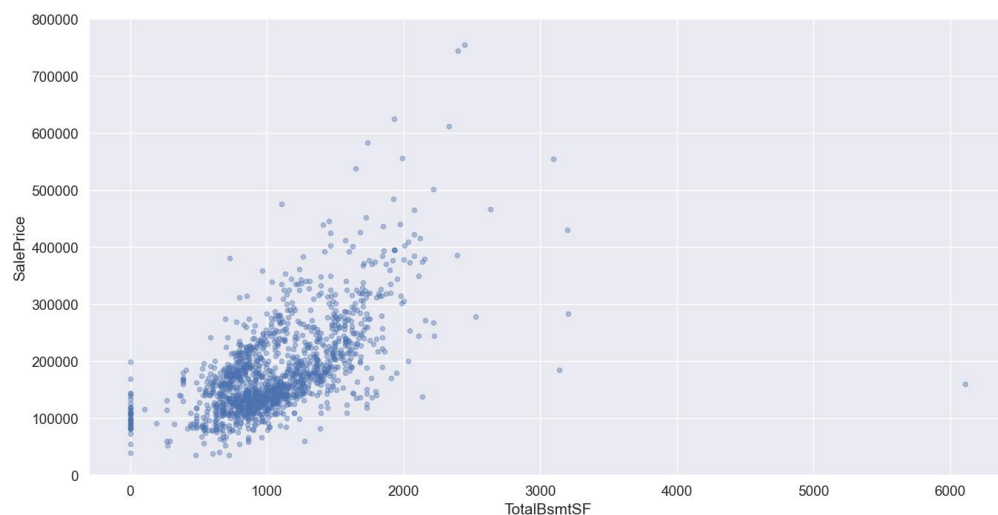
Mặc dù không phải là một xu hướng mạnh mẽ, nhưng ta thấy 'SalePrice' có xu hướng tăng với các mẫu có mới hơn là các ngôi nhà được xây từ lâu

TotalBsmstSF: Total square feet of basement area - Tổng diện tích của khu vực tầng hầm tính bằng feet vuông.

TotalBsmstSF: Total square feet of basement area - Tổng diện tích của khu vực tầng hầm tính bằng feet vuông.

```
var = 'TotalBsmstSF'
print(f'Hệ số tương quan giữa SalePrice và {var} ', df_train['SalePrice'].corr(df_train[var]))
data = pd.concat([df_train['SalePrice'], df_train[var]], axis=1)
data.plot.scatter(x='TotalBsmstSF', y='SalePrice', alpha=0.4, ylim=(0,800000));
#data.plot.scatter(x='TotalBsmstSF', y='SalePrice', ylim=(0,800000));
```

Hệ số tương quan giữa SalePrice và TotalBsmstSF 0.6135805515591956

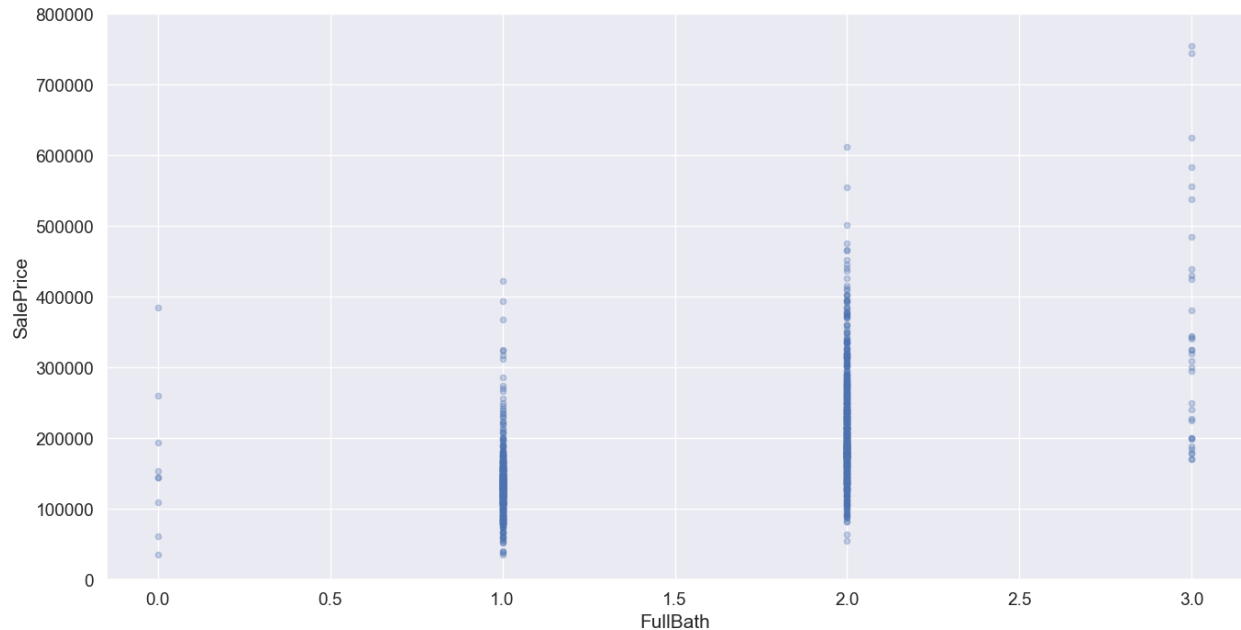


```

: var = 'FullBath'
print(f'Hệ số tương quan giữa SalePrice và {var} ', df_train['SalePrice'].corr(df_train[var]))
data = pd.concat([df_train['SalePrice'], df_train[var]], axis=1)
data.plot.scatter(x=var, y='SalePrice', alpha=0.3, ylim=(0,800000));

```

Hệ số tương quan giữa SalePrice và FullBath 0.5606637627484449

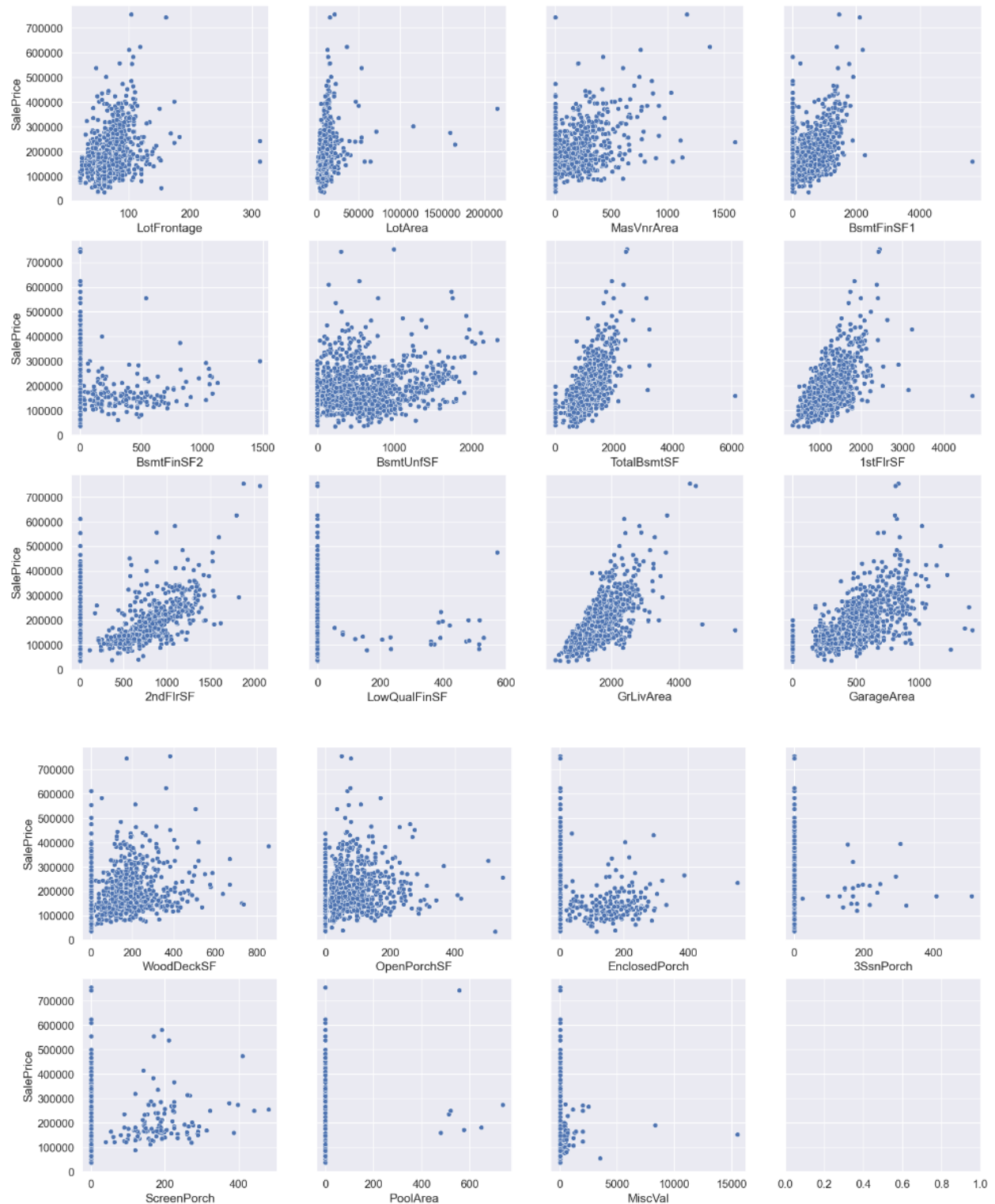


Các giá trị SalePrice phân tán khá rộng, đặc biệt là ở các giá trị FullBath cao hơn, điều này có thể cho thấy rằng giá nhà không chỉ phụ thuộc vào số lượng phòng tắm mà còn vào nhiều yếu tố khác.

FullBath có ý nghĩa trong việc dự đoán SalePrice và có thể giúp cải thiện độ chính xác của mô hình dự đoán khi kết hợp với các biến khác.

2.4 Trực quan hóa

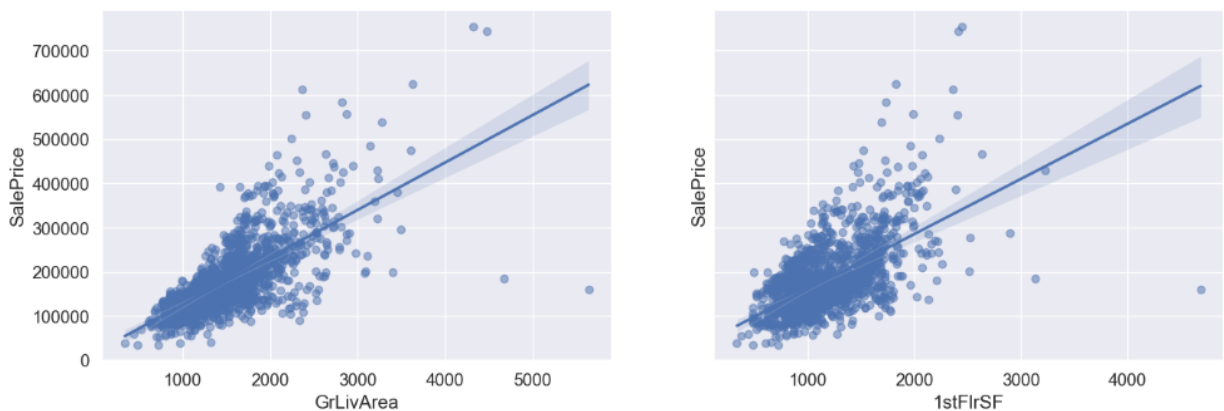
Phân phối giá trị của SalePrice với các biến số



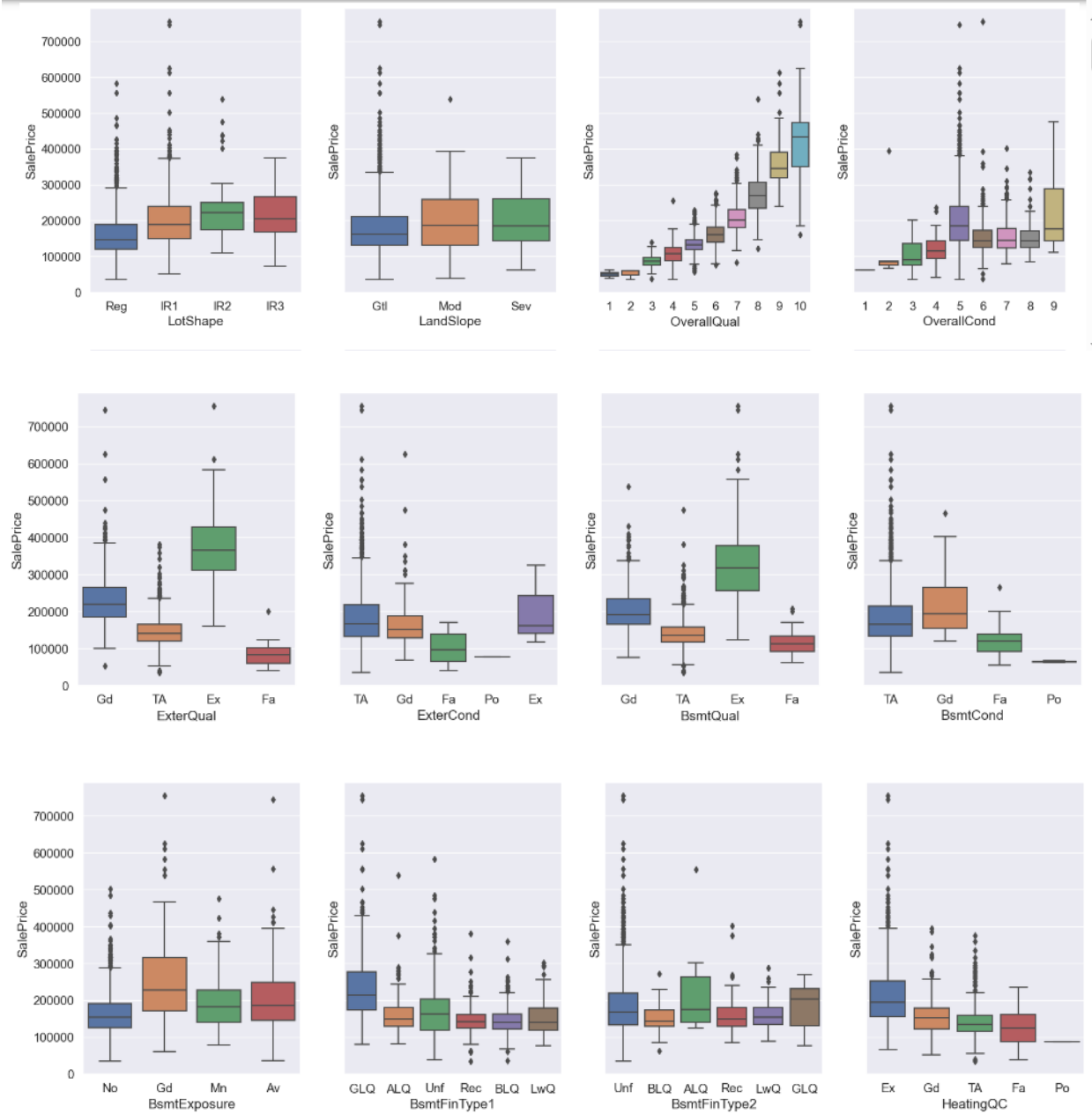
Có một số ngoại lệ trong biểu đồ của *LotFrontage*, *BsmtFinSF1*, *TotalBsmtSF*, *1stFlrSF*, *GrLivArea*, *EnclosedPorch*, *MiscVal*

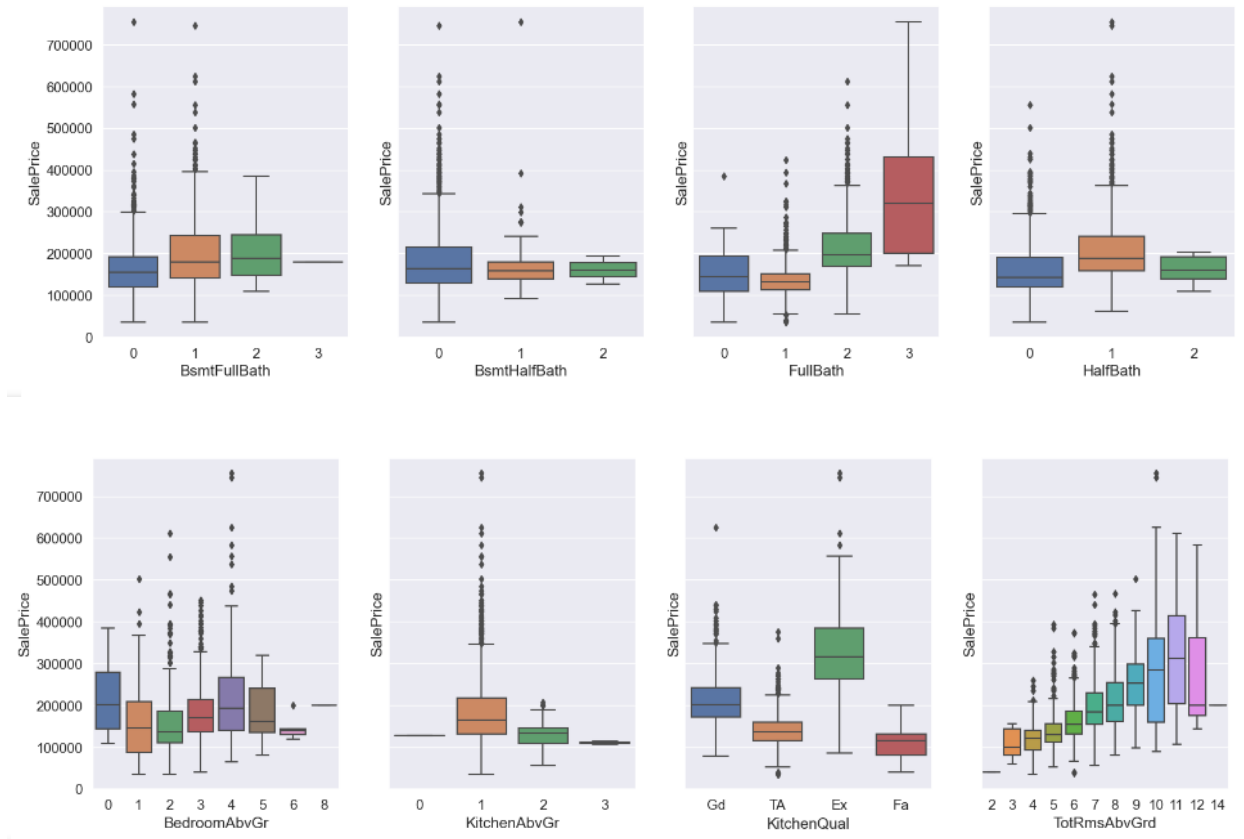
Trong số tất cả các đặc điểm số, *Ground Living Area (GrLivArea)* và *First Floor Square Feet (1stFlrSF)* có mối tương quan chặt chẽ với *SalePrice*

```
fig, axes = plt.subplots(1, 2, sharey=True, figsize=(16, 5))
axes = np.ravel(axes)
sns.regplot(data=df_train, x="GrLivArea", y="SalePrice", ax=axes[0], scatter_kws={'alpha':0.5})
sns.regplot(data=df_train, x="1stFlrSF", y="SalePrice", ax=axes[1], scatter_kws={'alpha':0.5})
plt.show()
```



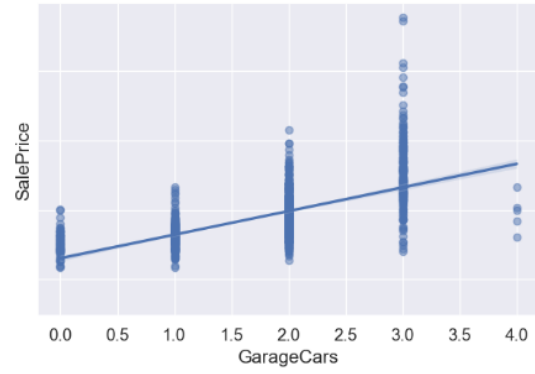
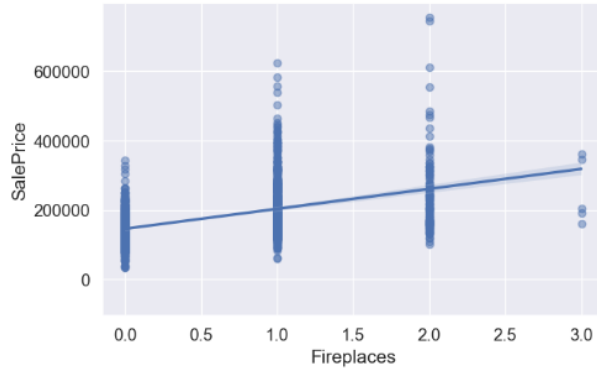
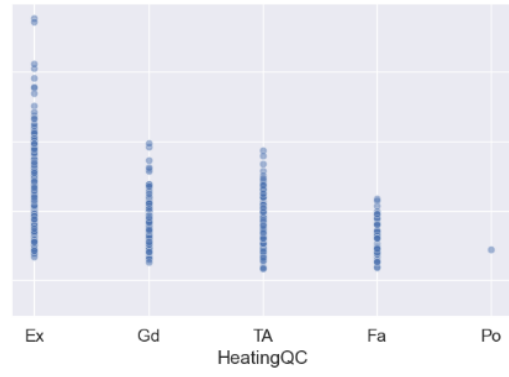
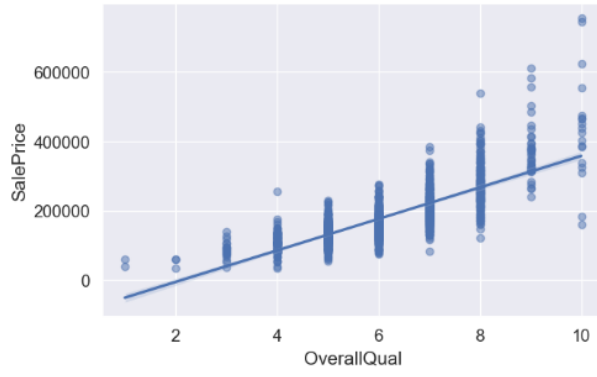
- *Ordinal features*



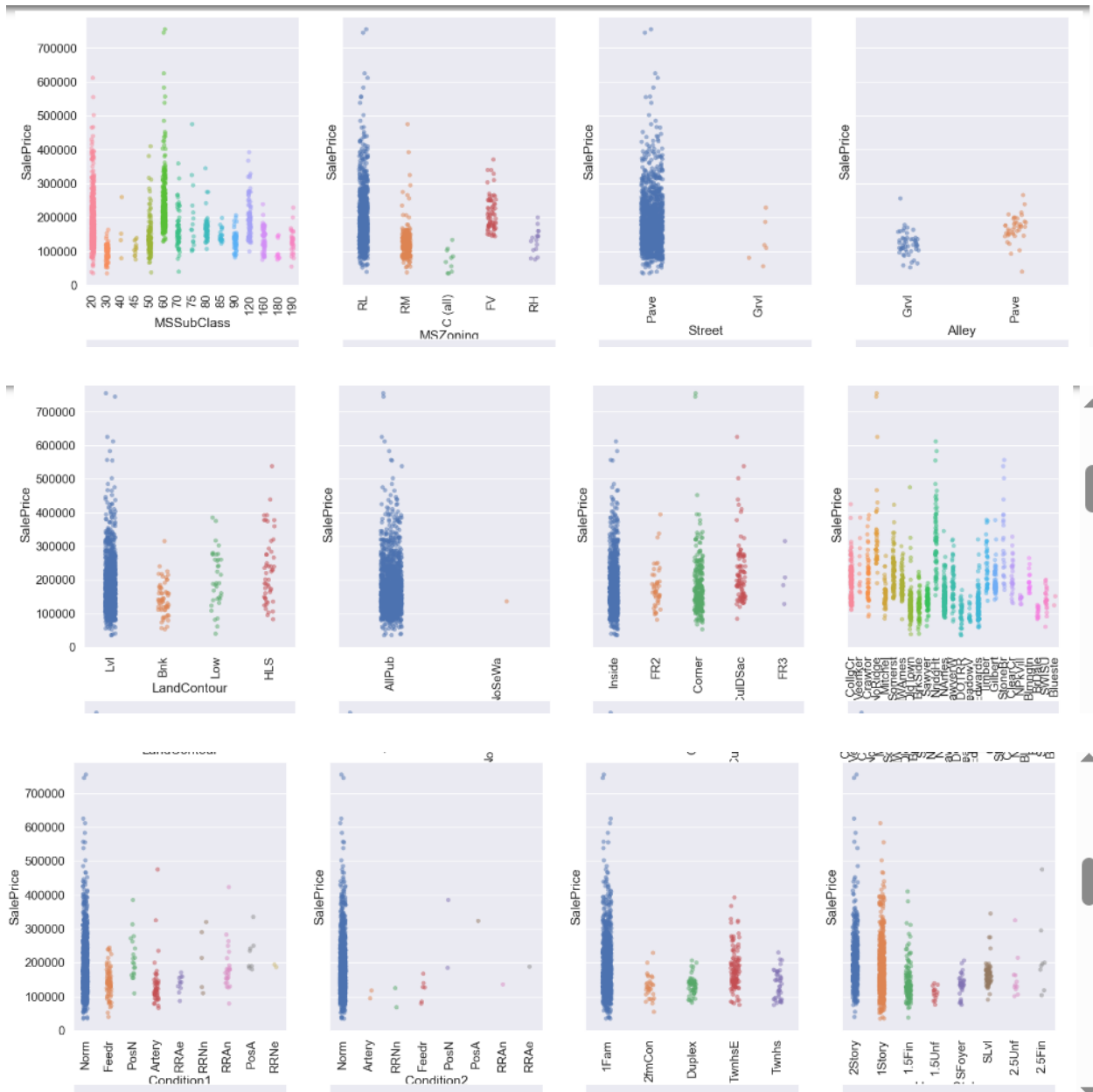


Trong số các thuộc tính thứ tự, Overall Quality (OverallQual) cho thấy mối tương quan chặt chẽ với SalePrice, tiếp theo là Heating quality and condition (HeatingQC), Fireplaces. Điều hợp lý là HeatingQC và Fireplaces có mối tương quan với SalePrice vì mùa đông ở Ames (từ tháng 12 đến tháng 2) lạnh giá, có tuyết và nhiều gió, trung bình là -5 độ C.

Kích thước của gara về sức chứa ô tô (GarageCars) cũng tương quan với Giá bán. Với sức chứa 4 chiếc, giá nhà giảm xuống, dường như người dân Ames hầu hết chỉ có tối đa 3 chiếc ô tô



- *Categorical features*



Nói chung về những biến phân loại này ta có:

- Loại đường vào khu nhà được trải nhựa.
- Nhà có đầy đủ tiện ích công cộng cần thiết: Điện, Gas, Nước
- Nhà có máy lạnh

Những ngôi nhà có giá cao hơn là:

- Nhà ở Khu dân cư Làng Nổi, trên 144K
- Những ngôi nhà ở khu vực lân cận Làng Northridge, Công viên Northridge Heights, trung bình 322K
- Nhà được lợp bằng mặt gạch, hoặc các loại gỗ khác nhau (Ván cứng, Ván ép, Ván lợp bằng gỗ)
- Những ngôi nhà được ốp đá veneer, trên 119k
- Nhà mới, kể cả nhà đã hoàn thiện một phần



3. Feature Engineering

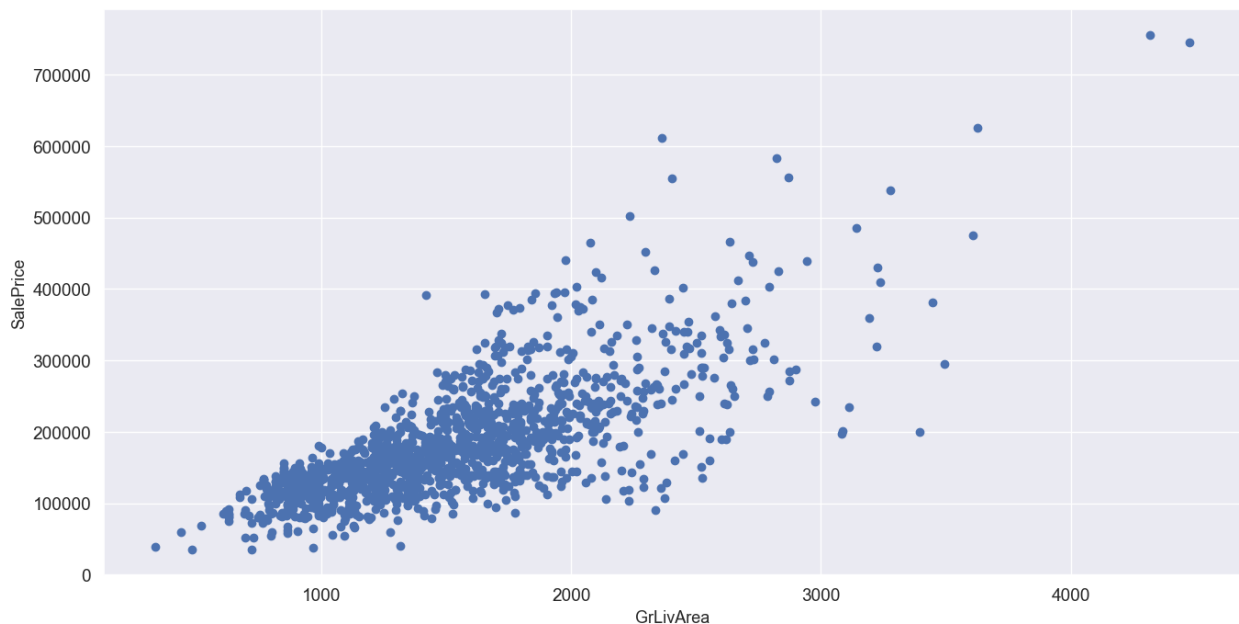
3.1 Xử lý ngoại lệ (Outlier)

Từ mục 2.3, ta có thể một số ngoại lệ (Outlier) ở các OverallQual/SalePrice và GrLivArea/SalePrice.

```
# Loại bỏ các ngoại lệ
df_train.drop(df_train[(df_train['OverallQual']<5) & (df_train['SalePrice']>200000)].index, inplace=True)
df_train.drop(df_train[(df_train['GrLivArea']>4500) & (df_train['SalePrice']<300000)].index, inplace=True)
df_train.reset_index(drop=True, inplace=True)
```

Kiểm tra lại sau khi loại bỏ các outlier

```
#Vẽ lại GrLivArea/SalePrice
fig, ax = plt.subplots()
ax.scatter(df_train['GrLivArea'], df_train['SalePrice'])
plt.ylabel('SalePrice', fontsize=13)
plt.xlabel('GrLivArea', fontsize=13)
plt.show()
```



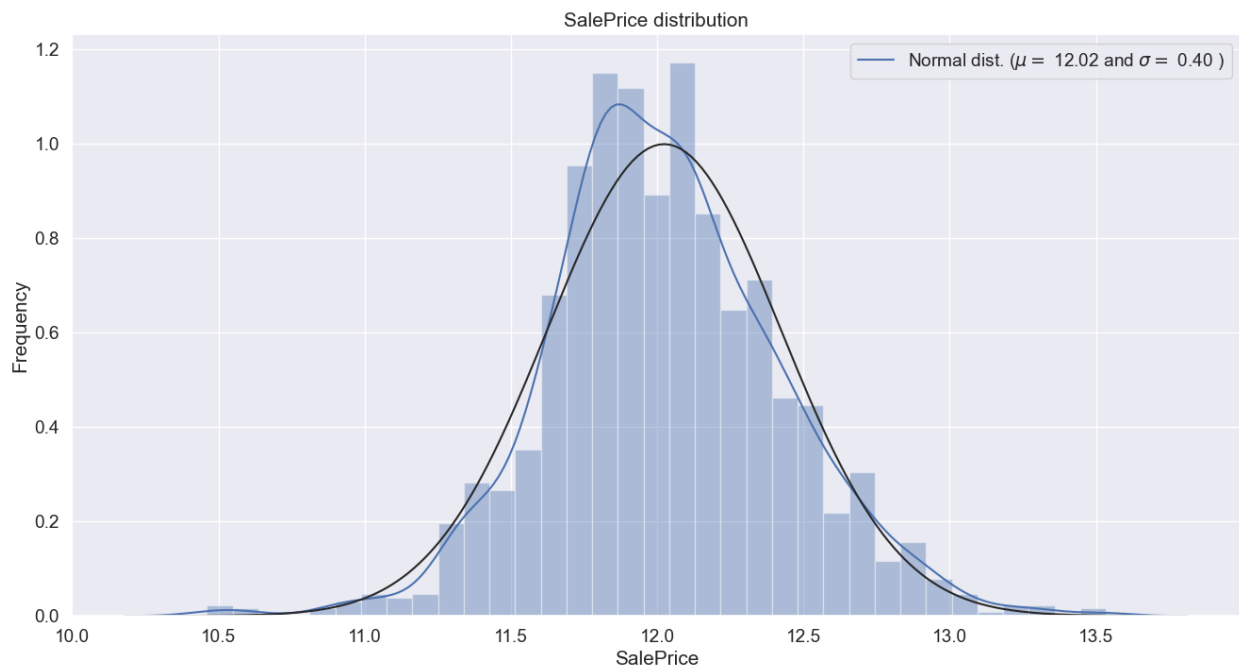
3.2 Log Transformation

Như đã phân tích ở mục 2.1, 'SalePrice' bị lệch sang phải. Đây là một vấn đề, vì hầu hết các mô hình ML không hoạt động tốt với dữ liệu không được phân phối chuẩn. Chúng ta phi tuyến tính hóa cho biến mục tiêu bằng cách áp dụng biến đổi ' $\log(1+x)$ ' (logarit tự nhiên cơ số e) để giảm độ lệch của phân phối và làm cho dữ liệu gần với phân phối chuẩn hơn, giúp cải thiện hiệu suất của các mô hình dự đoán.

```
# log(1+SalePrice) transform
df_train["SalePrice"] = np.log1p(df_train["SalePrice"])
```

```
#Kiểm tra phân phối mới
# xanh dương - đường cong phân phối chuẩn (normal distribution) để so sánh phân phối của dữ liệu đã biến đổi với phân phối chuẩn.
sns.distplot(df_train['SalePrice'], fit=norm);
# Lấy các tham số phù hợp được sử dụng
(mu, sigma) = norm.fit(df_train['SalePrice'])
print('\n Giá trị trung bình  $\mu$  hay mu = {:.2f} và độ lệch chuẩn của phân phối  $\sigma$  hay sigma = {:.2f}\n'.format(mu, sigma))

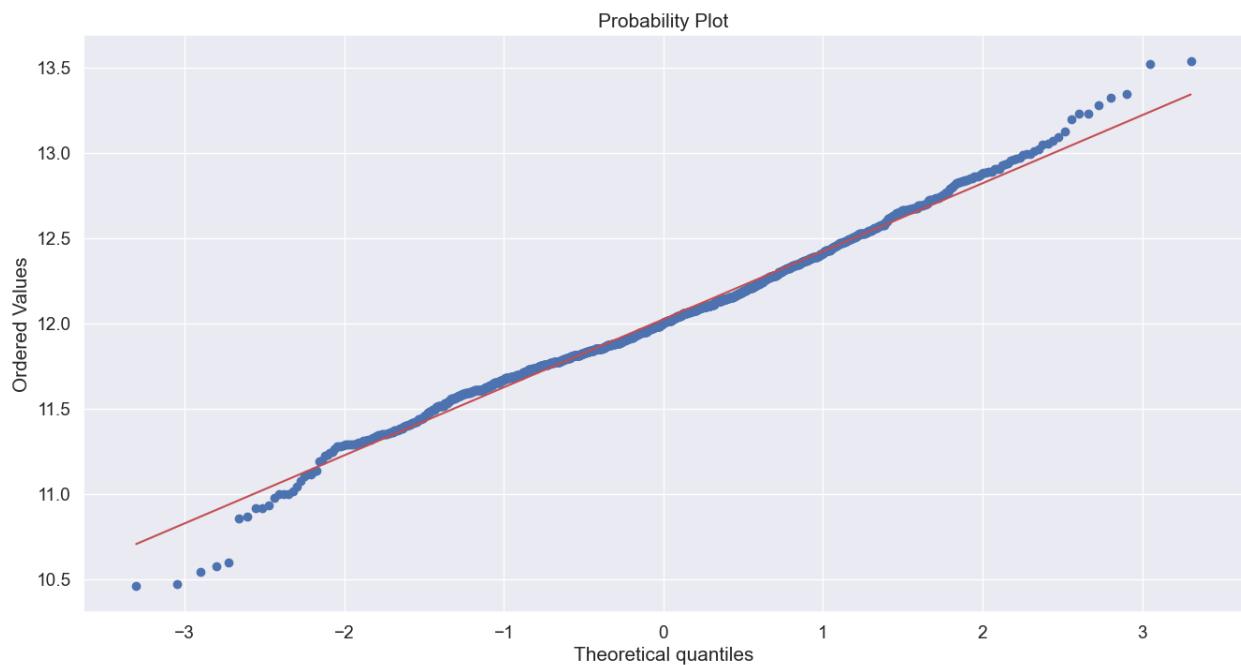
# Vẽ đồ thị phân phối
plt.legend(['Normal dist. ( $\mu$ = {:.2f} and  $\sigma$ = {:.2f} )'.format(mu, sigma)],
           loc='best')
plt.ylabel('Frequency')
plt.title('SalePrice distribution')
```



* Đường cong xanh dương: Ta thấy thấy rằng dữ liệu sau khi biến đổi có phân phối gần giống với phân phối chuẩn.

* Đường cong đen: hân phối thực tế của dữ liệu SalePrice sau khi biến đổi log. Nó cho thấy tần suất xuất hiện của các giá trị SalePrice đã biến đổi.

```
#Get also the QQ-plot (Quantile-Quantile plot)
fig = plt.figure()
res = stats.probplot(df_train['SalePrice'], plot=plt)
plt.show()
```



QQ-plot này cho thấy rằng sau khi áp dụng phép biến đổi $\log(1+\text{SalePrice})$, phân phối của biến SalePrice gần với phân phối chuẩn.

Các điểm xanh (SalePrice) nằm hầu như trên trục đỏ (phân phối chuẩn). Ở 2 đầu đồ thị có 1 vài điểm ngoại lệ lệch khỏi đường thẳng đỏ nhưng không ảnh hưởng nhiều đến phân phối tổng thể.

-

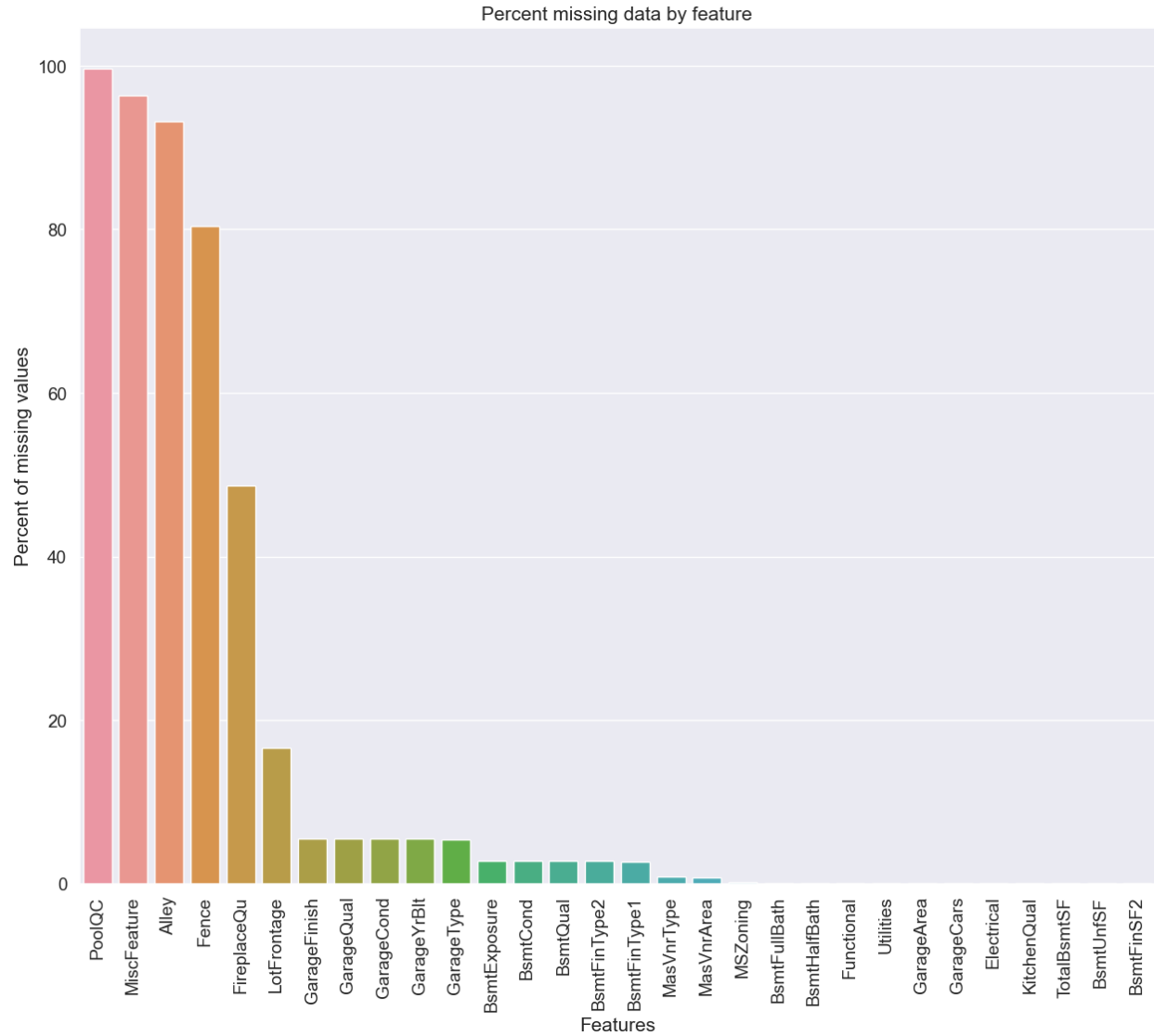
3.3 Missing Data and Imputation

3.3.1 Kiểm tra Missing Data

```
all_data_na = (all_data.isnull().sum() / len(all_data)) * 100
all_data_na = all_data_na.drop(all_data_na[all_data_na == 0].index).sort_values(ascending=False)[:30]
missing_data = pd.DataFrame({'Missing Ratio' :all_data_na})
missing_data.head(20)
```

	Missing Ratio
PoolQC	99.691358
MiscFeature	96.399177
Alley	93.209877
Fence	80.418381
FireplaceQu	48.696845
LotFrontage	16.632373
GarageFinish	5.452675
GarageQual	5.452675
GarageCond	5.452675
GarageYrBlt	5.452675
GarageType	5.384088
BsmtExposure	2.812071
BsmtCond	2.812071
BsmtQual	2.777778
BsmtFinType2	2.743484

```
f, ax = plt.subplots(figsize=(15, 12))
plt.xticks(rotation=90)
sns.barplot(x=all_data_na.index, y=all_data_na)
plt.xlabel('Features', fontsize=15)
plt.ylabel('Percent of missing values', fontsize=15)
plt.title('Percent missing data by feature', fontsize=15)
```



3.4.2 Imputating Missing Data

Chúng tôi impute chúng bằng cách tiến hành tuần tự các thuộc tính có giá trị bị thiếu

```
all_data["PoolQC"] = all_data["PoolQC"].fillna("None")
```

- **MiscFeature**: mô tả dữ liệu cho biết NA có nghĩa là "không có tính năng thêm"

```
all_data["MiscFeature"] = all_data["MiscFeature"].fillna("None")
```

37]

- **Alley**: mô tả dữ liệu cho biết NA có nghĩa là "không có ngõ vào"

+ Code + Markdown

```
all_data["Alley"] = all_data["Alley"].fillna("None")
```

- **Fence**: mô tả dữ liệu cho biết NA có nghĩa là "không có hàng rào"

```
all_data["Fence"] = all_data["Fence"].fillna("None")
```

+ Code + Markdown

Python

- **FireplaceQu**: mô tả dữ liệu cho biết NA có nghĩa là "không có lò sưởi"

```
all_data["FireplaceQu"] = all_data["FireplaceQu"].fillna("None")
```

Python

- **LotFrontage**: Vì diện tích của mỗi con đường nối với ngôi nhà có thể có diện tích tương tự với các ngôi nhà khác trong vùng lân cận của nó, nên chúng ta có thể điền các giá trị còn thiếu bằng LotFrontage trung bình của vùng lân cận.

```
# Nhóm theo vùng lân cận và điền giá trị còn thiếu theo LotFrontage trung bình của tất cả vùng lân cận
all_data["LotFrontage"] = all_data.groupby("Neighborhood")["LotFrontage"].transform(
    lambda x: x.fillna(x.median()))
```

Python

- **GarageType**, **GarageFinish**, **GarageQual** và **GarageCond**: Thay thế dữ liệu bị thiếu bằng None

```
for col in ('GarageType', 'GarageFinish', 'GarageQual', 'GarageCond'):
    all_data[col] = all_data[col].fillna('None')
```

- **GarageYrBlt**, **GarageArea** và **GarageCars**: Thay thế dữ liệu bị thiếu bằng 0 (Vì Không có gara = không có ô tô trong gara)

```
for col in ('GarageYrBlt', 'GarageArea', 'GarageCars'):
    all_data[col] = all_data[col].fillna(0)
```

- **BsmtFinSF1**, **BsmtFinSF2**, **BsmtUnfSF**, **TotalBsmtSF**, **BsmtFullBath** và **BsmtHalfBath**: các giá trị bị thiếu có thể bằng 0 vì không có tầng hầm

```
for col in ('BsmtFinSF1', 'BsmtFinSF2', 'BsmtUnfSF', 'TotalBsmtSF', 'BsmtFullBath', 'BsmtHalfBath'):
    all_data[col] = all_data[col].fillna(0)
```

- **BsmtQual**, **BsmtCond**, **BsmtExposure**, **BsmtFinType1** và **BsmtFinType2** : Đối với tất cả các tính năng phân loại liên quan đến tầng hầm này, NaN có nghĩa là không có tầng hầm.

```
for col in ('BsmtQual', 'BsmtCond', 'BsmtExposure', 'BsmtFinType1', 'BsmtFinType2'):
    all_data[col] = all_data[col].fillna('None')
```

Python

- **MasVnrArea** và **MasVnrType** : NA rất có thể có nghĩa là không có lớp vữa xây cho những ngôi nhà này. Chúng ta có thể điền 0 cho vùng (area) và None cho loại (type).

```
all_data["MasVnrType"] = all_data["MasVnrType"].fillna("None")
all_data["MasVnrArea"] = all_data["MasVnrArea"].fillna(0)
```

Python

- **MSZoning** (Phân loại phân vùng chung): 'RL' cho đến nay là giá trị phổ biến nhất. Vì vậy chúng ta có thể điền vào các giá trị còn thiếu bằng 'RL'

```
all_data['MSZoning'] = all_data['MSZoning'].fillna(all_data['MSZoning'].mode()[0])
```

Python

```
utilities_counts = all_data['Utilities'].value_counts()
# Hiển thị kết quả
print(utilities_counts)
```

Python

```
AllPub    2913
NoSeWa     1
Name: Utilities, dtype: int64
```

- **Utilities** : Đối với tính năng phân loại này, tất cả các bản ghi đều là "AllPub", ngoại trừ một "NoSeWa" và 2 NA. Vì ngôi nhà có 'NoSewa' nằm trong tập huấn luyện nên tính năng này sẽ không giúp ích gì trong việc lập mô hình dự đoán. Sau đó chúng ta có thể loại bỏ nó một cách an toàn.

```
all_data = all_data.drop(['Utilities'], axis=1)
```

Python

```
all_data = all_data.drop(['Utilities'], axis=1)
```

- **GarageArea** : Bỏ cột GarageArea như đã nói tại phần 3

```
all_data = all_data.drop(['GarageArea'], axis=1)
```

- **Functional** : mô tả dữ liệu cho biết NA có nghĩa là typical

```
all_data["Functional"] = all_data["Functional"].fillna("Typ")
```

- **Electrical** : Nó có một giá trị NA. Vì tính năng này chủ yếu có 'SBrkr' nên chúng tôi có thể đặt giá trị đó cho giá trị còn thiếu.

```
all_data['Electrical'] = all_data['Electrical'].fillna(all_data['Electrical'].mode()[0])
```


- **KitchenQual**: Chỉ có một giá trị NA và giống như Electrical, chúng tôi đặt 'TA' (giá trị thường xuyên nhất) cho giá trị còn thiếu trong KitchenQual.

```
all_data['KitchenQual'] = all_data['KitchenQual'].fillna(all_data['KitchenQual'].mode()[0])
```

- **Exterior1st** và **Exterior2nd**: Một lần nữa Cả Exterior 1 & 2 chỉ thiếu một giá trị. Chúng tôi sẽ chỉ thay thế bằng chuỗi phổ biến nhất.

```
all_data['Exterior1st'] = all_data['Exterior1st'].fillna(all_data['Exterior1st'].mode()[0])
all_data['Exterior2nd'] = all_data['Exterior2nd'].fillna(all_data['Exterior2nd'].mode()[0])
```

- **SaleType**: Điền lại với tần suất thường xuyên nhất là "WD"

```
all_data['SaleType'] = all_data['SaleType'].fillna(all_data['SaleType'].mode()[0])
```

- **MSSubClass**: Na rất có thể có nghĩa là Không có lớp xây dựng. Chúng ta có thể thay thế các giá trị bị thiếu bằng None.

[+ Code](#) [+ Markdown](#)

```
all_data['MSSubClass'] = all_data['MSSubClass'].fillna("None")
```

3.5 Transformation

Chuyển đổi một số biến số sang phân loại

```
#MSSubClass=Lớp tòa nhà
all_data['MSSubClass'] = all_data['MSSubClass'].apply(str)

#Chuyển đổi OverallCond thành biến phân loại
all_data['OverallCond'] = all_data['OverallCond'].astype(str)

#Năm và tháng bán được chuyển đổi thành các đặc trưng phân loại.
all_data['YrSold'] = all_data['YrSold'].astype(str)
all_data['MoSold'] = all_data['MoSold'].astype(str)
```

3.6 Label Encoding

Mã hóa nhãn cho một số biến phân loại có thể chứa thông tin trong tập hợp thứ tự của chúng.

```
from sklearn.preprocessing import LabelEncoder

cols = ('FireplaceQu', 'BsmtQual', 'BsmtCond', 'GarageQual', 'GarageCond',
        'ExterQual', 'ExterCond', 'HeatingQC', 'PoolQC', 'KitchenQual', 'BsmtFinType1',
        'BsmtFinType2', 'Functional', 'Fence', 'BsmtExposure', 'GarageFinish', 'LandSlope',
        'LotShape', 'PavedDrive', 'Street', 'Alley', 'CentralAir', 'MSSubClass', 'OverallCond',
        'YrSold', 'MoSold')
# Xử lý các cột, áp dụng LabelEncoder cho các đặc trưng phân loại
for c in cols:
    lbl = LabelEncoder()
    lbl.fit(list(all_data[c].values))
    all_data[c] = lbl.transform(list(all_data[c].values))

# in ra kích thước dữ liệu
print('Shape all_data: {}'.format(all_data.shape))
```

Shape all_data: (2916, 77)

3.7 Adding feature

Vì các đặc điểm liên quan đến diện tích rất quan trọng để xác định giá nhà nên chúng tôi bổ sung thêm một đặc điểm nữa đó là tổng diện tích tầng hầm, tầng một và tầng hai của mỗi ngôi nhà.

```
# Thêm thuộc tính tổng diện tích
all_data['TotalSF'] = all_data['TotalBsmtSF'] + all_data['1stFlrSF'] + all_data['2ndFlrSF']

all_data.info()
```

3.8 Skewed features and Box Cox Transformation of (highly) skewed features

```
numeric_feats = all_data.dtypes[all_data.dtypes != "object"].index

# Kiểm tra độ lệch của tất cả các thuộc tính số
skewed_feats = all_data[numeric_feats].apply(lambda x: skew(x.dropna())).sort_values(ascending=False)
print("\nSkew in numerical features: \n")
skewness = pd.DataFrame({'Skew' :skewed_feats})
skewness.head(10)
```

Skew in numerical features:

	Skew
MiscVal	21.935910
PoolArea	17.685603
LotArea	13.262550
LowQualFinSF	12.082427
3SsnPorch	11.370087
LandSlope	4.994554
KitchenAbvGr	4.299698
BsmtFinSF2	4.143683
EnclosedPorch	4.001570
ScreenPorch	3.944305

Chúng tôi sử dụng hàm scipy `'boxcox1p'` để tính toán phép biến đổi Box-Cox cho $\ln(1+x)$.

Lưu ý rằng việc đặt $\lambda=0$ tương đương với việc sử dụng hàm `'log1p'` (logarit tự nhiên cơ số e của $(1+x)$ đã được dùng ở trên cho biến mục tiêu.

```
skewness = skewness[abs(skewness) > 0.75]
print("Có {} đặc trưng số bị lệch cần được biến đổi Box Cox".format(skewness.shape[0]))

from scipy.special import boxcox1p
skewed_features = skewness.index
lam = 0.15
for feat in skewed_features:
    #all_data[feat] += 1 # Tăng giá trị mỗi đặc trưng lên 1 (được bình luận và không sử dụng)
    all_data[feat] = boxcox1p(all_data[feat], lam) # Áp dụng phép biến đổi Box-Cox với lambda = 0.15
```

Có 58 đặc trưng số bị lệch cần được biến đổi Box Cox

3.9 Dummy Feature

Tạo các đặc trưng phân loại dạng dummy Chuyển các biến phân loại về dạng nhị phân

[+ Code](#) [+ Markdown](#)

```
all_data = pd.get_dummies(all_data) #sẽ chuyển đổi mỗi giá trị duy nhất trong biến phân loại  
print(all_data.shape)
```

(2916, 219)

- Sau khi áp dụng `pd.get_dummies` trên DataFrame `all_data`, số lượng hàng sẽ không thay đổi nhưng số lượng cột sẽ tăng lên tương ứng với số lượng các giá trị phân loại duy nhất trong các biến phân loại.
- Ta thấy rõ ràng các cột phân loại đã được thay thế bằng các cột giá trị nhị phân của các biến duy nhất trong các cột phân loại được chuyển đổi

`test.head(5)`

Python

	MSSubClass	LotFrontage	LotArea	Street	Alley	LotShape	LandSlope	OverallQual	OverallCond	YearBuilt	...	SaleType_ConLw	SaleType_New	SaleType_Oth	SaleType_V
1457	2.055642	6.221214	20.479373	0.730463	0.730463	1.540963	0.0	2.055642	2.055642	14.121376	...	0	0	0	
1458	2.055642	6.244956	21.327220	0.730463	0.730463	0.000000	0.0	2.259674	2.055642	14.116605	...	0	0	0	
1459	2.885846	6.073289	21.196905	0.730463	0.730463	0.000000	0.0	2.055642	1.820334	14.178149	...	0	0	0	
1460	2.885846	6.172972	19.865444	0.730463	0.730463	0.000000	0.0	2.259674	2.055642	14.179714	...	0	0	0	
1461	0.000000	5.093857	17.257255	0.730463	0.730463	0.000000	0.0	2.602594	1.820334	14.170316	...	0	0	0	

5 rows × 219 columns

4. Lựa chọn mô hình phân tích và kết quả

Các thư viện cần thiết

```
from sklearn.linear_model import ElasticNet, Lasso, BayesianRidge, LassoLarsIC
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.kernel_ridge import KernelRidge
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import RobustScaler
from sklearn.base import BaseEstimator, TransformerMixin, RegressorMixin, clone
from sklearn.model_selection import KFold, cross_val_score, train_test_split
from sklearn.metrics import mean_squared_error
import xgboost as xgb
import lightgbm as lgb
#from sklearn.svm import SVR
#from catboost import CatBoostRegressor
```

4.1 Hàm kiểm tra

Sử dụng `cross_val_score` với K-fold cross-validation để đánh giá hiệu suất của mô hình trả về Root Mean Squared Error - RMSE - Đo lường sự khác biệt giữa giá trị dự đoán và giá trị thực tế trên

$$RMSE = \sqrt{\frac{\sum_{i=1}^N (Predicted_i - Actual_i)^2}{N}}$$

.RMSE có giá trị không âm và càng nhỏ càng tốt. Giá trị bằng 0 đạt được khi các dự đoán hoàn toàn chính xác.

```
n_folds = 5 # Chia dữ liệu thành 5 phần (folds) để thực hiện K-fold cross-validation.
# sử dụng cross_val_score lấy ra neg_mean_squared_error rồi trả về giá trị âm là mean squared error (MSE) rồi lấy square()
# trả về 1 mảng các giá trị RMSLE (Root Mean Squared Logarithmic Error) cho từng lần lặp của quá trình cross-validation.
def rmse_cv(model):
    kf = KFold(n_folds, shuffle=True, random_state=42).get_n_splits(train.values)
    rmse = np.sqrt(-cross_val_score(model, train.values, y_train, scoring="neg_mean_squared_error", cv=kf))
    return(rmse)
```

4.2 Xây dựng các mô hình

- Lasso: Sử dụng Lasso regression với `RobustScaler` để xử lý dữ liệu trước khi hồi quy vì Mô hình này có thể rất nhạy cảm với các ngoại lệ.
- ENet: ElasticNet regression với `RobustScaler`.
- KRR: Kernel Ridge Regression với kernel là hàm đa thức bậc 2.
- GBoost: Gradient Boosting Regressor. #sử dụng hubber loss để xử lý với các ngoại lệ

- Xgb: XGBoost Regressor.
- Lgb: LightGBM Regressor.

```
lasso = make_pipeline(RobustScaler(), Lasso(alpha=0.0005, random_state=1))
ENet = make_pipeline(RobustScaler(), ElasticNet(alpha=0.0005, l1_ratio=.9, random_state=3))
KRR = KernelRidge(alpha=0.6, kernel='polynomial', degree=2, coef0=2.5)
GBoost = GradientBoostingRegressor(n_estimators=3000, learning_rate=0.05, max_depth=4, max_features='sqrt',
                                   min_samples_leaf=15, min_samples_split=10, loss='huber', random_state=5)
model_xgb = xgb.XGBRegressor(colsample_bytree=0.4603, gamma=0.0468, learning_rate=0.05, max_depth=3, min_child_weight=1.7817,
                             n_estimators=2200, reg_alpha=0.4640, reg_lambda=0.8571, subsample=0.5213, silent=1, random_state=7,
                             bagging_freq=5, feature_fraction=0.2319, feature_fraction_seed=9, bagging_seed=9,
                             min_data_in_leaf=6, min_sum_hessian_in_leaf=11)
#model_rf = RandomForestRegressor(n_estimators=1000, random_state=42)
#model_svr = make_pipeline(RobustScaler(), SVR(kernel='linear'))
#model_cat = CatBoostRegressor(iterations=1000, learning_rate=0.05, depth=3, random_seed=42, silent=True)
```

4.3 Đánh giá các mô hình

```
score = rmse_cv(lasso)
print("\nLasso score: {:.4f} ({:.4f})\n".format(score.mean(), score.std()))
score = rmse_cv(ENet)
print("ElasticNet score: {:.4f} ({:.4f})\n".format(score.mean(), score.std()))
score = rmse_cv(KRR)
print("Kernel Ridge score: {:.4f} ({:.4f})\n".format(score.mean(), score.std()))
score = rmse_cv(GBoost)
print("Gradient Boosting score: {:.4f} ({:.4f})\n".format(score.mean(), score.std()))
score = rmse_cv(model_xgb)
print("Xgboost score: {:.4f} ({:.4f})\n".format(score.mean(), score.std()))
```

Lasso score: 0.1114 (0.0070)

ElasticNet score: 0.1115 (0.0070)

Kernel Ridge score: 0.1151 (0.0072)

Gradient Boosting score: 0.1164 (0.0073)

Xgboost score: 0.1165 (0.0070)

```
score = rmse_cv(model_lgb)
print("LGBM score: {:.4f} ({:.4f})\n".format(score.mean(), score.std()))

[LightGBM] [Warning] min_sum_hessian_in_leaf is set=11, min_child_weight=0.001 will be ignored. Current value: min_sum_hessian_in_leaf=11
[LightGBM] [Warning] bagging_fraction is set=0.8, subsample=1.0 will be ignored. Current value: bagging_fraction=0.8
[LightGBM] [Warning] bagging_freq is set=5, subsample_freq=0 will be ignored. Current value: bagging_freq=5
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.000635 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 1394
[LightGBM] [Info] Number of data points in the train set: 1166, number of used features: 174
[LightGBM] [Info] Start training from score 12.021438
[LightGBM] [Warning] min_data_in_leaf is set=6, min_child_samples=20 will be ignored. Current value: min_data_in_leaf=6
[LightGBM] [Warning] feature_fraction is set=0.2319, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=0.2319
[LightGBM] [Warning] min_sum_hessian_in_leaf is set=11, min_child_weight=0.001 will be ignored. Current value: min_sum_hessian_in_leaf=11
[LightGBM] [Warning] bagging_fraction is set=0.8, subsample=1.0 will be ignored. Current value: bagging_fraction=0.8
[LightGBM] [Warning] bagging_freq is set=5, subsample_freq=0 will be ignored. Current value: bagging_freq=5
LGBM score: 0.1147 (0.0073)
```

4.4 Averaging Models (Trung bình các mô hình)

- Tạo một lớp AveragingModels để trung bình dự đoán từ nhiều mô hình cơ sở khác nhau.
- Đánh giá hiệu suất của mô hình trung bình bằng hàm rmse_cv.

```
class AveragingModels(BaseEstimator, RegressorMixin, TransformerMixin):
    def __init__(self, models):
        self.models = models

    #tạo các bản sao (clones) của các mô hình và huấn luyện từng mô hình với dữ liệu huấn luyện (X, y).
    def fit(self, X, y):
        self.models_ = [clone(x) for x in self.models]
        for model in self.models_:
            model.fit(X, y)
        return self

    # dự đoán cho từng mô hình đã được huấn luyện và kết hợp các dự đoán bằng cách lấy trung bình các dự đoán đó.
    def predict(self, X):
        #kết hợp các dự đoán từ các mô hình thành một ma trận (mỗi cột là dự đoán từ một mô hình).
        predictions = np.column_stack([model.predict(X) for model in self.models_])
        # tính trung bình các dự đoán trên mỗi hàng (tức là trung bình các dự đoán từ các mô hình cho mỗi mẫu dữ liệu).
        return np.mean(predictions, axis=1)
```

ElasticNet , Gradient Boosting, Kernel Ridge Regression,lasso được truyền vào

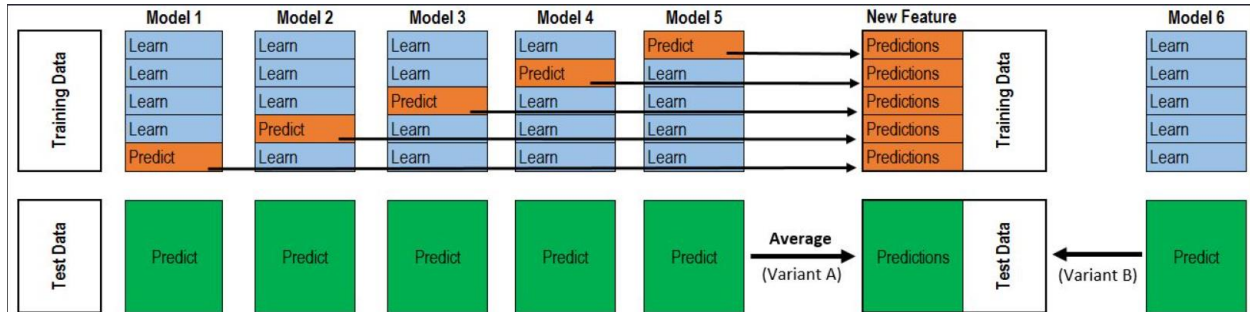
```
averaged_models = AveragingModels(models=(ENet, GBoost, KRR, lasso))
score = rmse_cv(averaged_models)
print("Averaged base models score: {:.4f} ({:.4f})\n".format(score.mean(), score.std()))

Averaged base models score: 0.1086 (0.0073)
```

Bằng cách kết hợp nhiều mô hình, chúng ta có thể giảm độ dao động của các dự đoán và cải thiện độ ổn định của mô hình.

Cách tiếp cận xếp chồng đơn giản nhất cũng thực sự cải thiện được điểm số, ta sẽ thử tạo ra 1 mô hình stacking mới

4.5 Stacking Model



Stacking là một phương pháp ensemble, trong đó nhiều mô hình cơ sở (base model) được huấn luyện

Các dự đoán từ các mô hình cơ sở được sử dụng làm đầu vào cho meta model để cải thiện độ chính xác của dự đoán. Meta model sau đó học cách kết hợp các dự đoán này để tạo ra dự đoán cuối cùng.

Tạo một lớp `StackingAveragedModels` để thực hiện stacking, kết hợp dự đoán từ nhiều mô hình cơ sở (base models) và sử dụng một mô hình meta (meta-model) để dự đoán cuối cùng

Đánh giá hiệu suất của mô hình trung bình bằng hàm `rmsle_cv`


```
# kế thừa từ BaseEstimator, RegressorMixin, và TransformerMixin của scikit-Learn, hoạt động giống như một mô hình hồi quy chuẩn.
class StackingAveragedModels(BaseEstimator, RegressorMixin, TransformerMixin):
    def __init__(self, base_models, meta_model, n_folds=5):
        self.base_models = base_models
        self.meta_model = meta_model
        self.n_folds = n_folds

    def fit(self, X, y):
        self.base_models_ = [list() for x in self.base_models]
        self.meta_model_ = clone(self.meta_model)
        # Sử dụng K-fold cross-validation để tạo ra các dự đoán "out-of-fold" từ các mô hình cơ sở.
        kfold = KFold(n_splits=self.n_folds, shuffle=True, random_state=156)

        out_of_fold_predictions = np.zeros((X.shape[0], len(self.base_models_)))
        for i, model in enumerate(self.base_models):
            for train_index, holdout_index in kfold.split(X, y):
                instance = clone(model) # Tạo các bản sao (clones) của các mô hình cơ sở và huấn luyện chúng trên tập dữ liệu.

                self.base_models_[i].append(instance)
                instance.fit(X[train_index], y[train_index])
                y_pred = instance.predict(X[holdout_index])
                out_of_fold_predictions[holdout_index, i] = y_pred

        # Đưa các dự đoán out_of_fold_prediction làm đầu vào để huấn luyện mô hình meta.
        self.meta_model_.fit(out_of_fold_predictions, y)
        return self

    def predict(self, X):
        # Thực hiện dự đoán cho từng mô hình cơ sở và kết hợp các dự đoán đó vào meta_features (mỗi cột là dự đoán từ một mô hình,
        meta_features = np.column_stack([np.column_stack([model.predict(X) for model in base_models]).mean(axis=1) for base_model in self.base_models_])
        # Sử dụng các dự đoán kết hợp làm đầu vào cho meta model để tạo ra dự đoán cuối cùng.
        return self.meta_model_.predict(meta_features)
```

- Base model: ElasticNet, Gradient Boosting, Kernel Ridge Regression
- Meta_model là Lasso

```
stacked_averaged_models = StackingAveragedModels(base_models=(ENet, GBoost, KRR), meta_model=lasso)
score = rmse_cv(stack_averaged_models)
print("Stacking Averaged models score: {:.4f} ({:.4f})".format(score.mean(), score.std()))
```

Stacking Averaged models score: 0.1080 (0.0069)

Ta thấy kết quả được cải thiện hơn khi ta bổ sung các meta learner

⇒ Ta thấy kết quả được cải thiện hơn khi ta bổ sung các meta learner

4.6 Huấn luyện và dự đoán các mô hình

Hàm đánh giá kết quả dự đoán

```
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error

def rmse(y, y_pred):
    return np.sqrt(mean_squared_error(y, y_pred))

def evaluate_model(y_true, y_pred):
    rmsle_score = np.sqrt(mean_squared_error(y_true, y_pred))
    r2 = r2_score(y_true, y_pred)
    mae = mean_absolute_error(y_true, y_pred)
    mse = mean_squared_error(y_true, y_pred)

    print(f"RMSE: {rmsle_score:.6f}")
    print(f"R^2: {r2:.4f}")
    print(f"MAE: {mae:.4f}")
    print(f"MSE: {mse:.4f}")
```

StackedRegressor:

```
stacked_averaged_models.fit(train.values, y_train)
stacked_train_pred = stacked_averaged_models.predict(train.values)
#Biến đổi các giá trị dự đoán từ log scale trở lại dạng ban đầu
stacked_pred = np.expml(stacked_averaged_models.predict(test.values))
print(rmse(y_train, stacked_train_pred))
```

0.07944129485161945

XGBoost

```
model_xgb.fit(train, y_train)
xgb_train_pred = model_xgb.predict(train)
#Biến đổi các giá trị dự đoán từ log scale trở lại dạng ban đầu
xgb_pred = np.expml(model_xgb.predict(test))
print(rmse(y_train, xgb_train_pred))
```

0.08704096001196739

LightGBM

```
model_lgb.fit(train, y_train)
lgb_train_pred = model_lgb.predict(train)
#Biến đổi các giá trị dự đoán từ log scale trở lại dạng ban đầu
lgb_pred = np.expml(model_lgb.predict(test.values))
print(rmse(y_train, lgb_train_pred))
```

[LightGBM] [Warning] min_data_in_leaf is set=6, min_child_samples=20 will be ignored. Current value: min_data_in_leaf=6
[LightGBM] [Warning] feature_fraction is set=0.2319, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=0.2319
[LightGBM] [Warning] min_sum_hessian_in_leaf is set=11, min_child_weight=0.001 will be ignored. Current value: min_sum_hessian_in_leaf=11
[LightGBM] [Warning] bagging_fraction is set=0.8, subsample=1.0 will be ignored. Current value: bagging_fraction=0.8
[LightGBM] [Warning] bagging_freq is set=5, subsample_freq=0 will be ignored. Current value: bagging_freq=5
[LightGBM] [Warning] Found whitespace in feature_names, replace with underlines
[LightGBM] [Warning] min_data_in_leaf is set=6, min_child_samples=20 will be ignored. Current value: min_data_in_leaf=6
[LightGBM] [Warning] feature_fraction is set=0.2319, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=0.2319
[LightGBM] [Warning] min_sum_hessian_in_leaf is set=11, min_child_weight=0.001 will be ignored. Current value: min_sum_hessian_in_leaf=11
[LightGBM] [Warning] bagging_fraction is set=0.8, subsample=1.0 will be ignored. Current value: bagging_fraction=0.8
[LightGBM] [Warning] bagging_freq is set=5, subsample_freq=0 will be ignored. Current value: bagging_freq=5
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.000476 seconds.
You can set `force\_row\_wise=true` to remove the overhead.
And if memory is not enough, you can set `force\_col\_wise=true`.
[LightGBM] [Info] Total Bins 1430
[LightGBM] [Info] Number of data points in the train set: 1457, number of used features: 180
[LightGBM] [Info] Start training from score 12.023721
[LightGBM] [Warning] min_data_in_leaf is set=6, min_child_samples=20 will be ignored. Current value: min_data_in_leaf=6
[LightGBM] [Warning] feature_fraction is set=0.2319, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=0.2319
[LightGBM] [Warning] min_sum_hessian_in_leaf is set=11, min_child_weight=0.001 will be ignored. Current value: min_sum_hessian_in_leaf=11
[LightGBM] [Warning] bagging_fraction is set=0.8, subsample=1.0 will be ignored. Current value: bagging_fraction=0.8
[LightGBM] [Warning] bagging_freq is set=5, subsample_freq=0 will be ignored. Current value: bagging_freq=5
[LightGBM] [Warning] min_data_in_leaf is set=6, min_child_samples=20 will be ignored. Current value: min_data_in_leaf=6
[LightGBM] [Warning] feature_fraction is set=0.2319, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=0.2319
[LightGBM] [Warning] min_sum_hessian_in_leaf is set=11, min_child_weight=0.001 will be ignored. Current value: min_sum_hessian_in_leaf=11
[LightGBM] [Warning] bagging_fraction is set=0.8, subsample=1.0 will be ignored. Current value: bagging_fraction=0.8
[LightGBM] [Warning] bagging_freq is set=5, subsample_freq=0 will be ignored. Current value: bagging_freq=5
0.07221819513300258

4.7 Tạo ra dự đoán cuối cùng bằng cách kết hợp

- Kết hợp các dự đoán từ các mô hình stacked_averaged_models, model_xgb, và model_lgb với các trọng số tương ứng để tạo ra dự đoán cuối cùng.
- Tìm ra các trọng số tối ưu cho từng mô hình trong một tập hợp các mô hình (ensemble) để tối thiểu hóa RMSLE (Root Mean Squared Logarithmic Error)

```
print('RMSE score on train data:')
print(rmse(y_train, stacked_train_pred*0.70 + xgb_train_pred*0.15 + lgb_train_pred*0.15))
```

RMSE score on train data:
0.07739066720579575

Hàm ensemble_predict tìm các trọng số tối ưu cho từng mô hình

```
: # tìm các trọng số tối ưu cho từng mô hình
def ensemble_predict(X_train, y_train, X_valid, y_valid, models):
    # Dự đoán dùng từng model
    pred1 = models[0].predict(X_valid)
    pred2 = models[1].predict(X_valid)
    pred3 = models[2].predict(X_valid)

    #kết hợp các dự đoán từ các mô hình thành một ma trận (mỗi cột là dự đoán từ một mô hình).
    preds = np.column_stack((pred1, pred2, pred3))

    # Tính toán RMSE
    def rmsle_ensemble(weights):
        final_pred = np.sum(preds * weights, axis=1)
        return np.sqrt(mean_squared_error(y_valid, final_pred)) #Tính RMSLE giữa giá trị thực tế (y_valid) và giá trị dự đoán kết hợp

    # Tìm ra các trọng số tốt nhất ( best weights)
    from scipy.optimize import minimize
    weights = np.ones(3) / 3 # Initial guess for the weights
    constraints = ({'type': 'eq', 'fun': lambda w: 1 - sum(w)}) #ràng buộc tổng các trọng số phải bằng 1.
    bounds = [(0, 1)] * 3 #Giới hạn cho các trọng số, đảm bảo rằng mỗi trọng số nằm trong khoảng từ 0 đến 1.

    #Tìm các trọng số tốt nhất bằng cách sử dụng hàm tối thiểu hóa từ scipy để tìm các trọng số tốt nhất
    # method='SLSQP' chỉ định phương pháp tối ưu sử dụng là Sequential Least Squares Programming
    result = minimize(rmsle_ensemble, weights, method='SLSQP', bounds=bounds, constraints=constraints)

    return result.x
```

```
: models = [stacked_averaged_models, model_xgb, model_lgb]
#ở đây ta dùng tập train để train và validation
best_weights = ensemble_predict(train.values, y_train, train.values, y_train, models)
print(f"Best weights: {best_weights}")

# Use the best weights to make the final prediction
final_train_pred = stacked_train_pred * best_weights[0] + xgb_train_pred * best_weights[1] + lgb_train_pred * best_weights[2]

# RMSLE
print('Best RMSE score on train data:')
#print(rmsle(y_train, final_train_pred))
evaluate_model(y_train, final_train_pred)

[LightGBM] [Warning] min_data_in_leaf is set=6, min_child_samples=20 will be ignored. Current value: min_data_in_leaf=6
[LightGBM] [Warning] feature_fraction is set=0.2319, colsample_bytree=1.0 will be ignored. Current value: feature_fraction=0.2319
[LightGBM] [Warning] min_sum_hessian_in_leaf is set=11, min_child_weight=0.001 will be ignored. Current value: min_sum_hessian_in_leaf=11
[LightGBM] [Warning] bagging_fraction is set=0.8, subsample=1.0 will be ignored. Current value: bagging_fraction=0.8
[LightGBM] [Warning] bagging_freq is set=5, subsample_freq=0 will be ignored. Current value: bagging_freq=5
Best weights: [9.02124249e-02 2.08166817e-16 9.09787575e-01]
Best RMSE score on train data:
RMSE: 0.072139
R^2: 0.9674
MAE: 0.0511
MSE: 0.0052
```

* *RMSE (Root Mean Squared Error):*

Đo lường sự khác biệt giữa giá trị dự đoán và giá trị thực tế.

Giá trị cho ra là thấp nhất trong các mô hình đã tạo ra .

*** R^2 (Coefficient of Determination):**

Đo lường mức độ mà các biến dự đoán giải thích được sự biến đổi của biến mục tiêu.

Giá trị nằm trong khoảng từ 0 đến 1. Kết quả 0.9674 cho thấy mô hình đã rất hiệu quả

*** MAE (Mean Absolute Error):**

Sai số trung bình giữa giá trị dự đoán và giá trị thực tế. Giá trị thấp chứng tỏ mô hình dự đoán hiệu quả.

*** MSE (Mean Squared Error):**

Đo lường trung bình bình phương sai số giữa giá trị dự đoán và giá trị thực tế. Giá trị rất thấp cho thấy mô hình dự đoán tốt.

```
: final_pred = stacked_pred * best_weights[0] + xgb_pred * best_weights[1] + lgb_pred * best_weights[2]
sub = pd.DataFrame()
sub['Id'] = test_ID
sub['SalePrice'] = final_pred
sub.to_csv('submission.csv', index=False)
sub.head(5)
```

```
:
```

	Id	SalePrice
0	1461	120968.433565
1	1462	158540.068535
2	1463	184641.991163
3	1464	197958.697758
4	1465	190682.471503

TAI LIỆU THAM KHẢO

- [1] Tài liệu slide giảng dạy, cung cấp bởi GVHD Ths. Nguyễn Văn Thành.
- [2] Owid *Covid-19-data/public/data at master ovid/covid-19-DATA*, GitHub. Available at: <https://github.com/owid/covid-19-data/tree/master/public/data> (Accessed: 04 May 2023).
- [3] Minewiskan (no date) *Defining a data source, Defining a Data Source | Microsoft Learn*. Available at: <https://learn.microsoft.com/en-us/analysis-services/multidimensional-tutorial/lesson-1-2-defining-a-data-source?view=asallproducts-allversions> (Accessed: 04 May 2023).
- [4] Chugugrace (no date) *Integration Services (SSIS) projects and solutions - SQL server integration services (SSIS), SQL Server Integration Services (SSIS) | Microsoft Learn*. Available at: <https://learn.microsoft.com/en-us/sql/integration-services/integration-services-ssis-projects-and-solutions?view=sql-server-ver16> (Accessed: 04 May 2023).
- [5] Minewiskan (no date b) *Multidimensional Model Databases (ssas), Microsoft Learn*. Available at: <https://learn.microsoft.com/en-us/analysis-services/multidimensional-models/multidimensional-model-databases-ssas?view=asallproducts-allversions> (Accessed: 09 May 2023).